

islr-ch7

Rohit Goswami

+++ title = "ISLR :: Moving Beyond Linearity" author = ["Rohit Goswami"] date = 2020-02-19T09:47:00+00:00 tags = ["solutions", "R", "ISLR"] categories = ["programming"] draft = false toc = true comments = true +++

Chapter VII - Moving Beyond Linearity

All the questions are as per the ISL [seventh printing of the First edition](#)^[^fn:1].

Common

```
libsUsed<-c("dplyr","ggplot2","tidyverse",
            "ISLR","caret","MASS", "gridExtra",
            "pls","latex2exp","data.table")
invisible(lapply(libsUsed, library, character.only = TRUE))
```

Question 7.6 - Page 299

In this exercise, you will further analyze the `Wage` data set considered throughout this chapter.

- (a) Perform polynomial regression to predict `wage` using `age`. Use cross-validation to select the optimal degree d for the polynomial. What degree was chosen, and how does this compare to the results of hypothesis testing using ANOVA? Make a plot of the resulting polynomial fit to the data.
- (b) Fit a step function to predict `wage` using `age`, and perform cross-validation to choose the optimal number of cuts. Make a plot of the fit obtained. In this exercise, we will generate simulated data, and will then use this data to perform best subset selection.

Answer

Lets get the data.

```
set.seed(1984)
wageDat<-ISLR::Wage
wageDat %>% str %>% print
```

```
## 'data.frame':   3000 obs. of  11 variables:
## $ year      : int   2006 2004 2003 2003 2005 2008 2009 2008 2006 2004 ...
## $ age       : int   18 24 45 43 50 54 44 30 41 52 ...
## $ maritl    : Factor w/ 5 levels "1. Never Married",...: 1 1 2 2 4 2 2 1 1 2 ...
## $ race      : Factor w/ 4 levels "1. White","2. Black",...: 1 1 1 3 1 1 4 3 2 1 ...
## $ education : Factor w/ 5 levels "1. < HS Grad",...: 1 4 3 4 2 4 3 3 3 2 ...
## $ region    : Factor w/ 9 levels "1. New England",...: 2 2 2 2 2 2 2 2 2 2 ...
## $ jobclass   : Factor w/ 2 levels "1. Industrial",...: 1 2 1 2 2 2 1 2 2 2 ...
## $ health     : Factor w/ 2 levels "1. <=Good","2. >=Very Good": 1 2 1 2 1 2 2 1 2 2 ...
## $ health_ins : Factor w/ 2 levels "1. Yes","2. No": 2 2 1 1 1 1 1 1 1 1 ...
## $ logwage    : num   4.32 4.26 4.88 5.04 4.32 ...
## $ wage       : num   75 70.5 131 154.7 75 ...
## NULL
```

```
wageDat %>% summary %>% print
```

```
##      year      age      maritl      race
## Min.   :2003   Min.   :18.00   1. Never Married: 648   1. White:2480
## 1st Qu.:2004   1st Qu.:33.75   2. Married   :2074   2. Black: 293
## Median :2006   Median :42.00   3. Widowed   : 19    3. Asian: 190
## Mean    :2006   Mean    :42.41   4. Divorced   : 204   4. Other:  37
## 3rd Qu.:2008   3rd Qu.:51.00   5. Separated   : 55
## Max.    :2009   Max.    :80.00
##
##      education      region      jobclass
## 1. < HS Grad      :268   2. Middle Atlantic   :3000   1. Industrial :1544
## 2. HS Grad        :971   1. New England     :  0   2. Information:1456
## 3. Some College   :650   3. East North Central:  0
## 4. College Grad   :685   4. West North Central:  0
## 5. Advanced Degree:426   5. South Atlantic    :  0
##                      6. East South Central:  0
##                      (Other)              :  0
##
##      health      health_ins      logwage      wage
## 1. <=Good      : 858   1. Yes:2083   Min.    :3.000   Min.    : 20.09
## 2. >=Very Good:2142   2. No : 917   1st Qu.:4.447   1st Qu.: 85.38
##                      Median :4.653   Median :104.92
##                      Mean    :4.654   Mean    :111.70
##                      3rd Qu.:4.857   3rd Qu.:128.68
##                      Max.    :5.763   Max.    :318.34
##
```

```
wageDat %>% sapply(unique) %>% sapply(length) %>% print
```

```
##      year      age      maritl      race education      region      jobclass
##      7         61         5         4         5         1         2
##      health health_ins logwage      wage
##      2         2         508         508
```

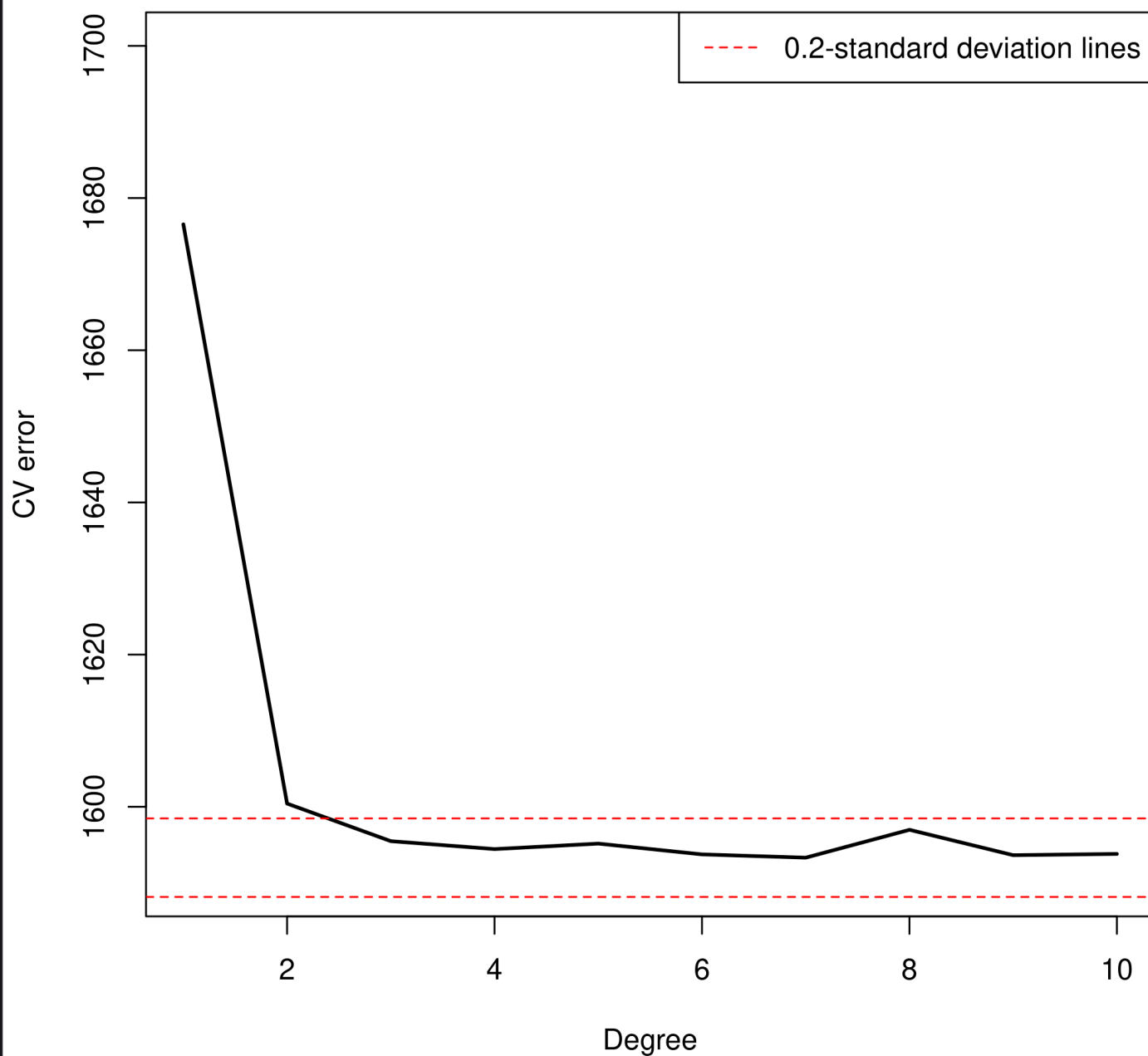
```
library(boot)
```

```
##
## Attaching package: 'boot'
```

```
## The following object is masked from 'package:lattice':
##
##      melanoma
```

a) Polynomial regression

```
all.deltas = rep(NA, 10)
for (i in 1:10) {
  glm.fit = glm(wage~poly(age, i), data=Wage)
  all.deltas[i] = cv.glm(Wage, glm.fit, K=10)$delta[2]
}
plot(1:10, all.deltas, xlab="Degree", ylab="CV error", type="l", pch=20, lwd=2, ylim=c(1590, 1700))
min.point = min(all.deltas)
sd.points = sd(all.deltas)
abline(h=min.point + 0.2 * sd.points, col="red", lty="dashed")
abline(h=min.point - 0.2 * sd.points, col="red", lty="dashed")
legend("topright", "0.2-standard deviation lines", lty="dashed", col="red")
```

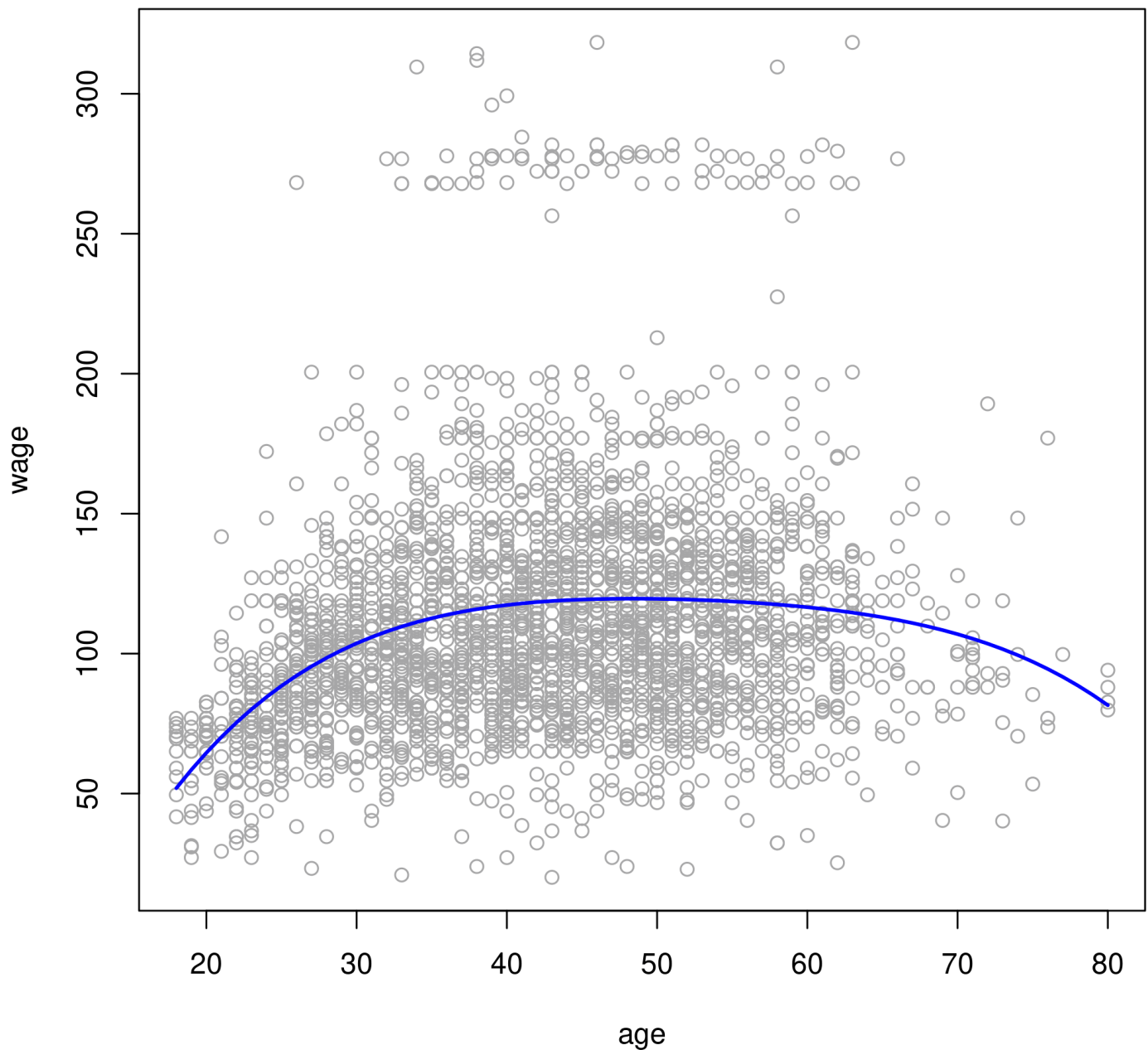


```
# ANOVA
fits=list()
for (i in 1:10) {
  fits[[i]]=glm(wage~poly(age,i),data=wageDat)
}
anova(fits[[1]],fits[[2]],fits[[3]],fits[[4]],fits[[5]],
      fits[[6]],fits[[7]],fits[[8]],fits[[9]],fits[[10]])
```

```
## Analysis of Deviance Table
##
## Model 1: wage ~ poly(age, i)
## Model 2: wage ~ poly(age, i)
## Model 3: wage ~ poly(age, i)
## Model 4: wage ~ poly(age, i)
## Model 5: wage ~ poly(age, i)
## Model 6: wage ~ poly(age, i)
## Model 7: wage ~ poly(age, i)
## Model 8: wage ~ poly(age, i)
## Model 9: wage ~ poly(age, i)
## Model 10: wage ~ poly(age, i)
##      Resid. Df Resid. Dev Df Deviance
## 1      2998      5022216
## 2      2997      4793430  1    228786
## 3      2996      4777674  1    15756
## 4      2995      4771604  1     6070
## 5      2994      4770322  1     1283
## 6      2993      4766389  1     3932
## 7      2992      4763834  1     2555
## 8      2991      4763707  1       127
## 9      2990      4756703  1     7004
## 10     2989      4756701  1         3
```

- The 4th degree looks the best at the moment

```
# 3rd or 4th degrees look best based on ANOVA test
# let's go with 4th degree fit
plot(wage~age, data=wageDat, col="darkgrey")
age.lims = range(wageDat$age)
age.grid = seq(from=age.lims[1], to=age.lims[2])
lm.fit = lm(wage~poly(age, 4), data=wageDat)
lm.pred = predict(lm.fit, data.frame(age=age.grid))
lines(age.grid, lm.pred, col="blue", lwd=2)
```



b) Step function and cross-validation

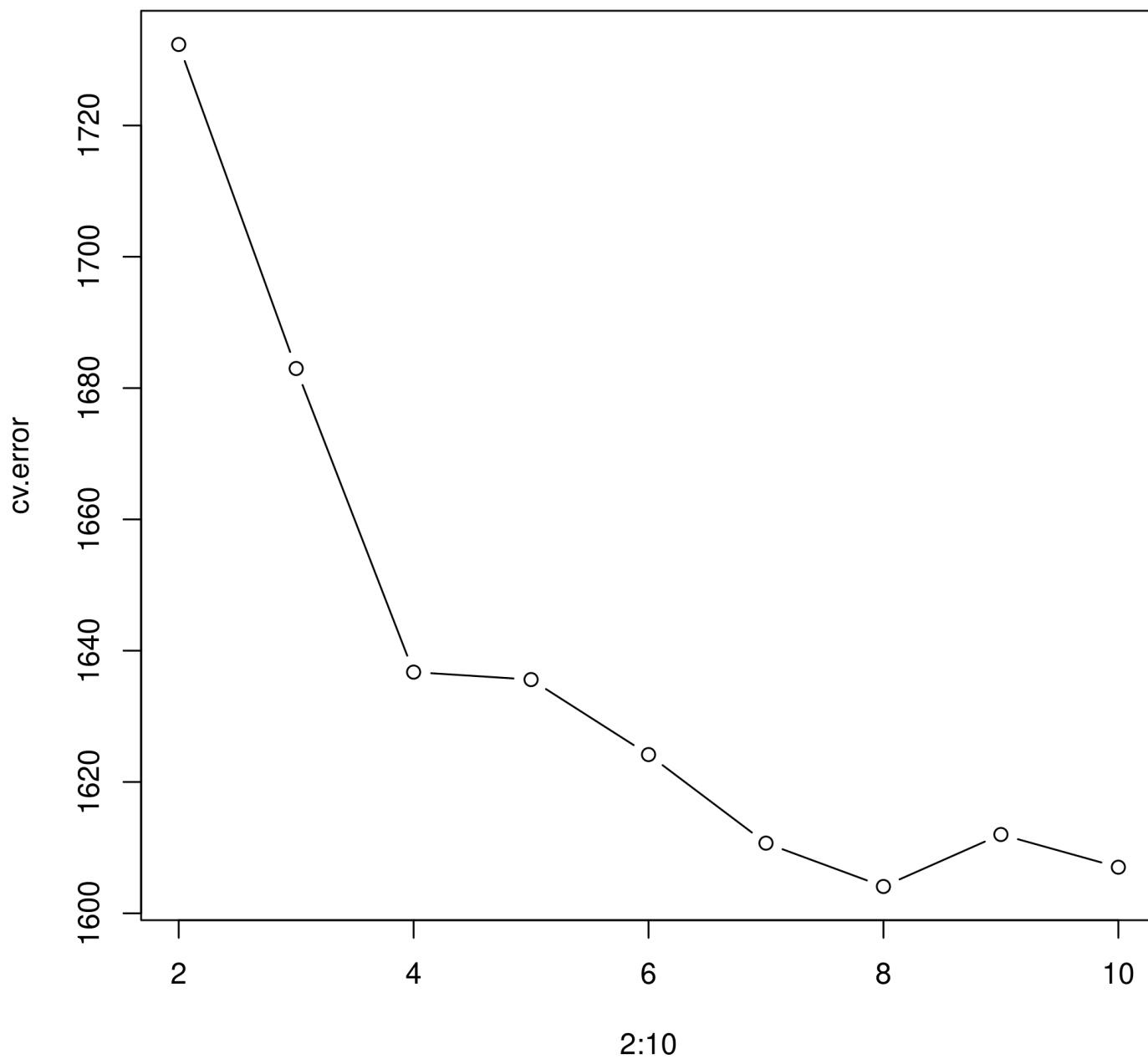
```
# cross-validation
cv.error <- rep(0,9)
for (i in 2:10) {
  wageDat$age.cut <- cut(wageDat$age,i)
  glm.fit <- glm(wage~age.cut, data=wageDat)
  cv.error[i-1] <- cv.glm(wageDat, glm.fit, K=10)$delta[1] # [1]:std, [2]:bias-corrected
}
cv.error
```

```
## [1] 1732.337 1682.978 1636.736 1635.600 1624.174 1610.688 1604.081 1612.005
## [9] 1607.022
```

```
cv.error
```

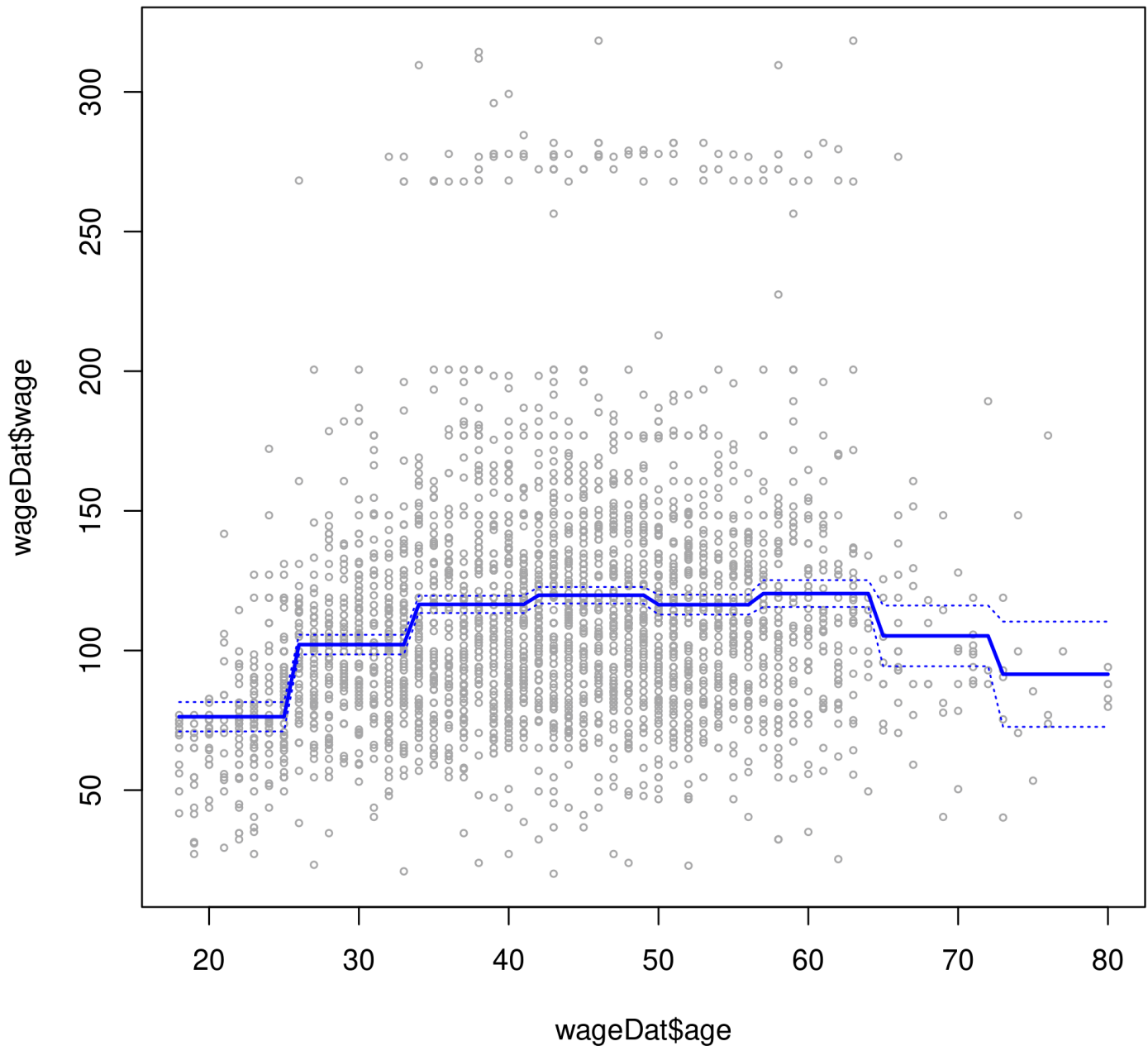
```
## [1] 1732.337 1682.978 1636.736 1635.600 1624.174 1610.688 1604.081 1612.005
## [9] 1607.022
```

```
plot(2:10, cv.error, type="b")
```



```
cut.fit <- glm(wage~cut(age,8), data=wageDat)
preds <- predict(cut.fit, newdata=list(age=age.grid), se=TRUE)
se.bands <- preds$fit + cbind(2*preds$se.fit, -2*preds$se.fit)
plot(wageDat$age, wageDat$wage, xlim=age.lims, cex=0.5, col="darkgrey")
title("Fit with 8 Age Bands")
lines(age.grid, preds$fit, lwd=2, col="blue")
matlines(age.grid, se.bands, lwd=1, col="blue", lty=3)
```

Fit with 8 Age Bands



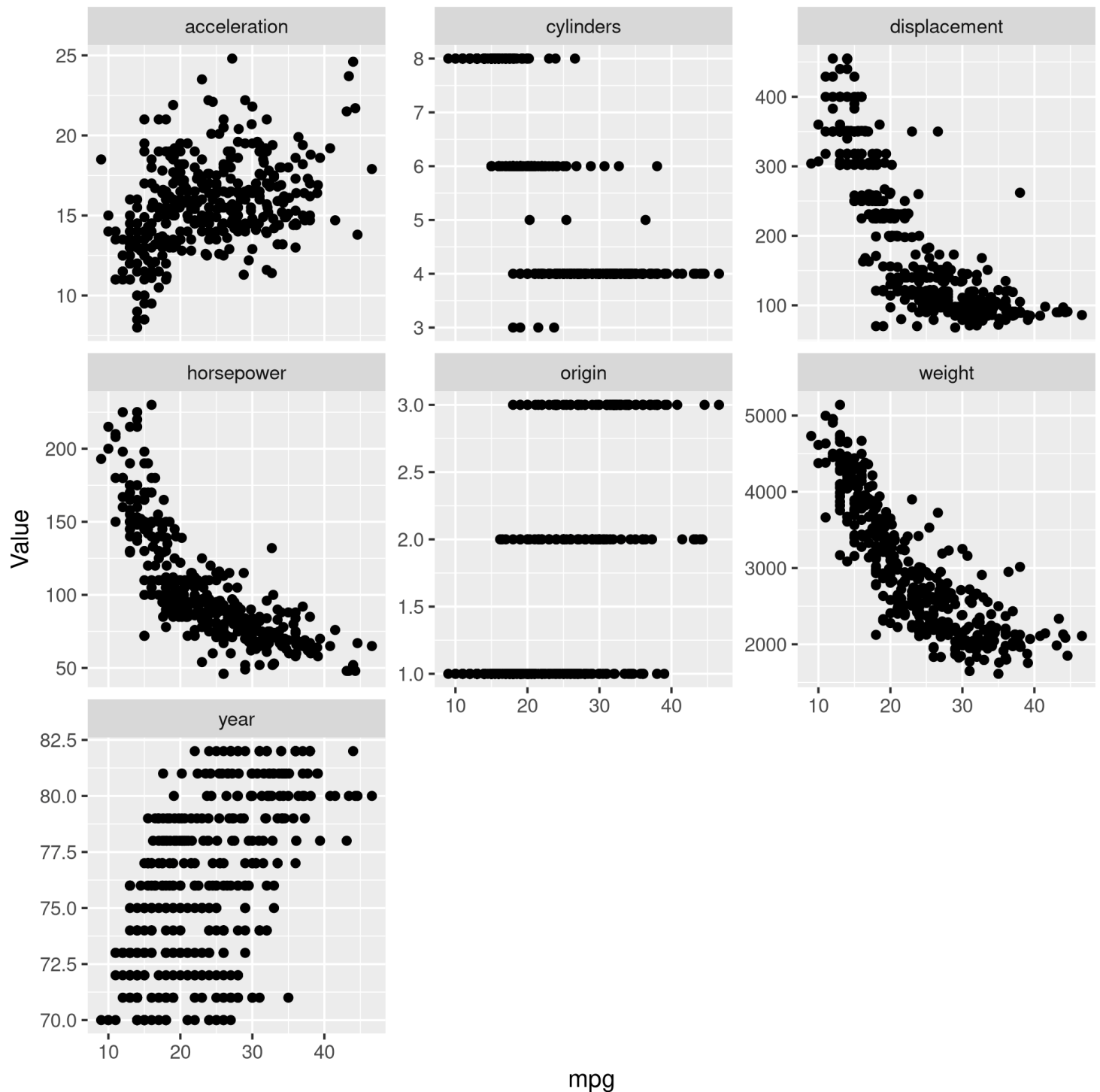
Question 7.8 - Page 299

Fit some of the non-linear models investigated in this chapter to the `Auto` data set. Is there evidence for non-linear relationships in this data set? Create some informative plots to justify your answer.

Answer

```
autoDat<- ISLR::Auto
```

```
autoDat %>% pivot_longer(-c(mpg,name),names_to="Params",values_to="Value") %>% ggplot(aes(x=mpg,y=Value)) +  
  geom_point() +  
  facet_wrap(~ Params, scales = "free_y")
```



Very clearly there is a lot of non-linearity in the `mpg` data, especially for `acceleration`, `weight`, `displacement`, `horsepower`.

```

rss = rep(NA, 10)
fits = list()
for (d in 1:10) {
  fits[[d]] = lm(mpg ~ poly(displacement, d), data = autoDat)
  rss[d] = deviance(fits[[d]])
}
rss %>% print

## [1] 8378.822 7412.263 7392.322 7391.722 7380.838 7270.746 7089.716 6917.401
## [9] 6737.801 6610.190

anova(fits[[1]],fits[[2]],fits[[3]],fits[[4]],fits[[5]],
      fits[[6]],fits[[7]],fits[[8]],fits[[9]],fits[[10]])

```



```
## Analysis of Variance Table
##
## Model 1: mpg ~ poly(displacement, d)
## Model 2: mpg ~ poly(displacement, d)
## Model 3: mpg ~ poly(displacement, d)
## Model 4: mpg ~ poly(displacement, d)
## Model 5: mpg ~ poly(displacement, d)
## Model 6: mpg ~ poly(displacement, d)
## Model 7: mpg ~ poly(displacement, d)
## Model 8: mpg ~ poly(displacement, d)
## Model 9: mpg ~ poly(displacement, d)
## Model 10: mpg ~ poly(displacement, d)
##      Res.Df    RSS Df Sum of Sq      F      Pr(>F)
## 1         390 8378.8
## 2         389 7412.3   1    966.56 55.7108 5.756e-13 ***
## 3         388 7392.3   1     19.94  1.1494  0.284364
## 4         387 7391.7   1      0.60  0.0346  0.852549
## 5         386 7380.8   1     10.88  0.6273  0.428823
## 6         385 7270.7   1    110.09  6.3455  0.012177 *
## 7         384 7089.7   1    181.03 10.4343  0.001344 **
## 8         383 6917.4   1    172.31  9.9319  0.001753 **
## 9         382 6737.8   1    179.60 10.3518  0.001404 **
## 10        381 6610.2   1    127.61  7.3553  0.006990 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Confirming our visual indications, we see that the second degree models work well.

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##   expand, pack, unpack
```

```
## Loaded glmnet 3.0-2
```

```
library(boot)
```

```
cv.errs = rep(NA, 15)
for (d in 1:15) {
  fit = glm(mpg ~ poly(displacement, d), data = Auto)
  cv.errs[d] = cv.glm(Auto, fit, K = 15)$delta[2]
}
which.min(cv.errs)
```

```
## [1] 10
```

Strangely, we seem to have ended up with a ten variable model here.

```
# Step functions
cv.errs = rep(NA, 10)
for (c in 2:10) {
  Auto$dis.cut = cut(Auto$displacement, c)
  fit = glm(mpg ~ dis.cut, data = Auto)
  cv.errs[c] = cv.glm(Auto, fit, K = 10)$delta[2]
}
which.min(cv.errs) %>% print
```

```
## [1] 9
```

```
library(splines)
cv.errs = rep(NA, 10)
for (df in 3:10) {
  fit = glm(mpg ~ ns(displacement, df = df), data = Auto)
  cv.errs[df] = cv.glm(Auto, fit, K = 10)$delta[2]
}
which.min(cv.errs) %>% print
```

```
## [1] 10
```

```
library(gam)
```

```
## Loading required package: foreach
```

```
##
## Attaching package: 'foreach'
```

```
## The following objects are masked from 'package:purrr':
##
##   accumulate, when
```

```
## Loaded gam 1.16.1
```

```
# GAMs
fit = gam(mpg ~ s(displacement, 4) + s(horsepower, 4), data = Auto)
```

```
## Warning in model.matrix.default(mt, mf, contrasts): non-list contrasts argument
## ignored
```

```
summary(fit)
```

```
##
## Call: gam(formula = mpg ~ s(displacement, 4) + s(horsepower, 4), data = Auto)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -11.2982  -2.1592  -0.4394   2.1247  17.0946
##
## (Dispersion Parameter for gaussian family taken to be 15.3543)
##
##      Null Deviance: 23818.99 on 391 degrees of freedom
## Residual Deviance: 5880.697 on 382.9999 degrees of freedom
## AIC: 2194.05
##
## Number of Local Scoring Iterations: 2
##
## Anova for Parametric Effects
##              Df  Sum Sq Mean Sq F value    Pr(>F)
## s(displacement, 4)    1 15254.9 15254.9 993.524 < 2e-16 ***
## s(horsepower, 4)      1  1038.4   1038.4  67.632 3.1e-15 ***
## Residuals           383   5880.7     15.4
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##              Npar Df  Npar F      Pr(F)
## (Intercept)
## s(displacement, 4)      3 13.613 1.863e-08 ***
## s(horsepower, 4)       3 15.606 1.349e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Question 7.9 - Pages 299-300

This question uses the variables `dis` (the weighted mean of distances to five Boston employment centers) and `nox` (nitrogen oxides concentration in parts per 10 million) from the `Boston` data. We will treat `dis` as the predictor and `nox` as the response.

- (a) Use the `poly()` function to fit a cubic polynomial regression to predict `nox` using `dis`. Report the regression output, and plot the resulting data and polynomial fits.
- (b) Plot the polynomial fits for a range of different polynomial degrees (say, from 1 to 10), and report the associated residual sum of squares.
- (c) Perform cross-validation or another approach to select the optimal degree for the polynomial, and explain your results.
- (d) Use the `bs()` function to fit a regression spline to predict `nox` using `dis`. Report the output for the fit using four degrees of freedom. How did you choose the knots? Plot the resulting fit.
- (e) Now fit a regression spline for a range of degrees of freedom, and plot the resulting fits and report the resulting RSS. Describe the results obtained.
- (f) Perform cross-validation or another approach in order to select the best degrees of freedom for a regression spline on this data. Describe your results.

Answer

```
boston<-MASS::Boston
boston %>% str %>% print
```

```
## 'data.frame':   506 obs. of  14 variables:
##  $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
##  $ zn     : num  18 0 0 0 0 12.5 12.5 12.5 12.5 ...
##  $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 ...
##  $ chas   : int   0 0 0 0 0 0 0 0 0 ...
##  $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 ...
##  $ rm     : num  6.58 6.42 7.18 7 7.15 ...
##  $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
##  $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
##  $ rad    : int   1 2 2 3 3 3 5 5 5 ...
##  $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
##  $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
##  $ black  : num  397 397 393 395 397 ...
##  $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
##  $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
## NULL
```

```
boston %>% summary %>% print
```

```
##      crim      zn      indus      chas
## Min.   : 0.00632   Min.   : 0.00   Min.   : 0.46   Min.   :0.00000
## 1st Qu.: 0.08204   1st Qu.: 0.00   1st Qu.: 5.19   1st Qu.:0.00000
## Median : 0.25651   Median : 0.00   Median : 9.69   Median :0.00000
## Mean   : 3.61352   Mean   : 11.36   Mean   :11.14   Mean   :0.06917
## 3rd Qu.: 3.67708   3rd Qu.: 12.50   3rd Qu.:18.10   3rd Qu.:0.00000
## Max.   :88.97620   Max.   :100.00   Max.   :27.74   Max.   :1.00000
##      nox      rm      age      dis
## Min.   :0.3850   Min.   :3.561   Min.   : 2.90   Min.   : 1.130
## 1st Qu.:0.4490   1st Qu.:5.886   1st Qu.: 45.02   1st Qu.: 2.100
## Median :0.5380   Median :6.208   Median : 77.50   Median : 3.207
## Mean   :0.5547   Mean   :6.285   Mean   : 68.57   Mean   : 3.795
## 3rd Qu.:0.6240   3rd Qu.:6.623   3rd Qu.: 94.08   3rd Qu.: 5.188
## Max.   :0.8710   Max.   :8.780   Max.   :100.00   Max.   :12.127
##      rad      tax      ptratio      black
## Min.   : 1.000   Min.   :187.0   Min.   :12.60   Min.   : 0.32
## 1st Qu.: 4.000   1st Qu.:279.0   1st Qu.:17.40   1st Qu.:375.38
## Median : 5.000   Median :330.0   Median :19.05   Median :391.44
## Mean   : 9.549   Mean   :408.2   Mean   :18.46   Mean   :356.67
## 3rd Qu.:24.000   3rd Qu.:666.0   3rd Qu.:20.20   3rd Qu.:396.23
## Max.   :24.000   Max.   :711.0   Max.   :22.00   Max.   :396.90
##      lstat      medv
## Min.   : 1.73   Min.   : 5.00
## 1st Qu.: 6.95   1st Qu.:17.02
## Median :11.36   Median :21.20
## Mean   :12.65   Mean   :22.53
## 3rd Qu.:16.95   3rd Qu.:25.00
## Max.   :37.97   Max.   :50.00
```

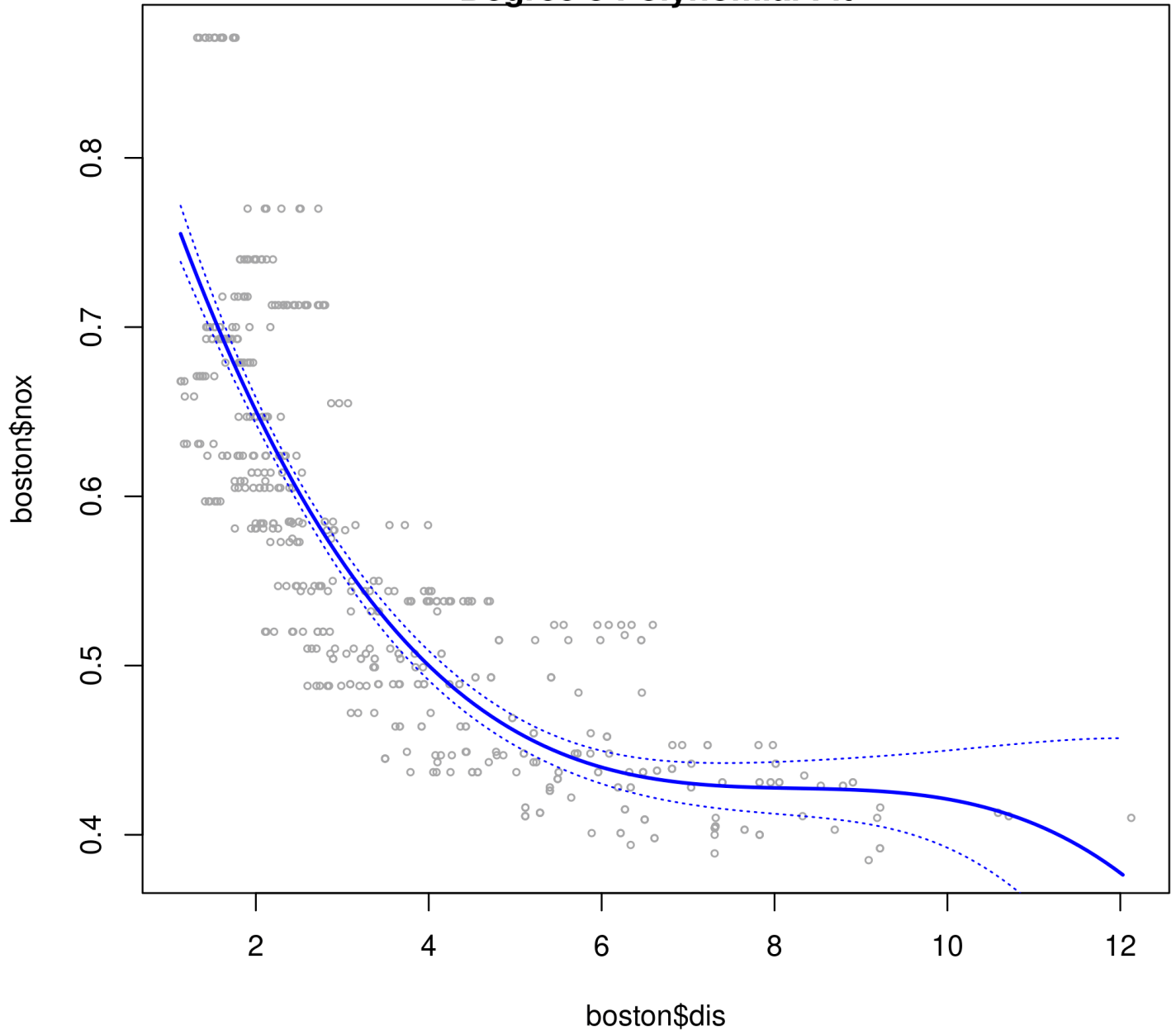
```
boston %>% sapply(unique) %>% sapply(length) %>% print
```

```
##      crim      zn      indus      chas      nox      rm      age      dis      rad      tax
##      504      26      76          2      81      446      356      412      9      66
## ptratio      black      lstat      medv
##      46      357      455      229
```

a) Polynomial

```
fit.03 <- lm(nox~poly(dis,3), data=boston)
dislims <- range(boston$dis)
dis.grid <- seq(dislims[1], dislims[2], 0.1)
preds <- predict(fit.03, newdata=list(dis=dis.grid), se=TRUE)
se.bands <- preds$fit + cbind(2*preds$se.fit, -2*preds$se.fit)
par(mfrow=c(1,1), mar=c(4.5,4.5,1,1), oma=c(0,0,4,0))
plot(boston$dis, boston$nox, xlim=dislims, cex=0.5, col="darkgrey")
title("Degree 3 Polynomial Fit")
lines(dis.grid, preds$fit, lwd=2, col="blue")
matlines(dis.grid, se.bands, lwd=1, col="blue", lty=3)
```

Degree 3 Polynomial Fit



```
summary(fit.03)
```

```
##
## Call:
## lm(formula = nox ~ poly(dis, 3), data = boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.121130 -0.040619 -0.009738  0.023385  0.194904
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.554695   0.002759  201.021 < 2e-16 ***
## poly(dis, 3)1 -2.003096   0.062071 -32.271 < 2e-16 ***
## poly(dis, 3)2  0.856330   0.062071  13.796 < 2e-16 ***
## poly(dis, 3)3 -0.318049   0.062071  -5.124 4.27e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06207 on 502 degrees of freedom
## Multiple R-squared:  0.7148, Adjusted R-squared:  0.7131
## F-statistic: 419.3 on 3 and 502 DF,  p-value: < 2.2e-16
```

b) Multiple Polynomials

```

rss.error <- rep(0,10)
for (i in 1:10) {
  lm.fit <- lm(nox~poly(dis,i), data=boston)
  rss.error[i] <- sum(lm.fit$residuals^2)
}
rss.error

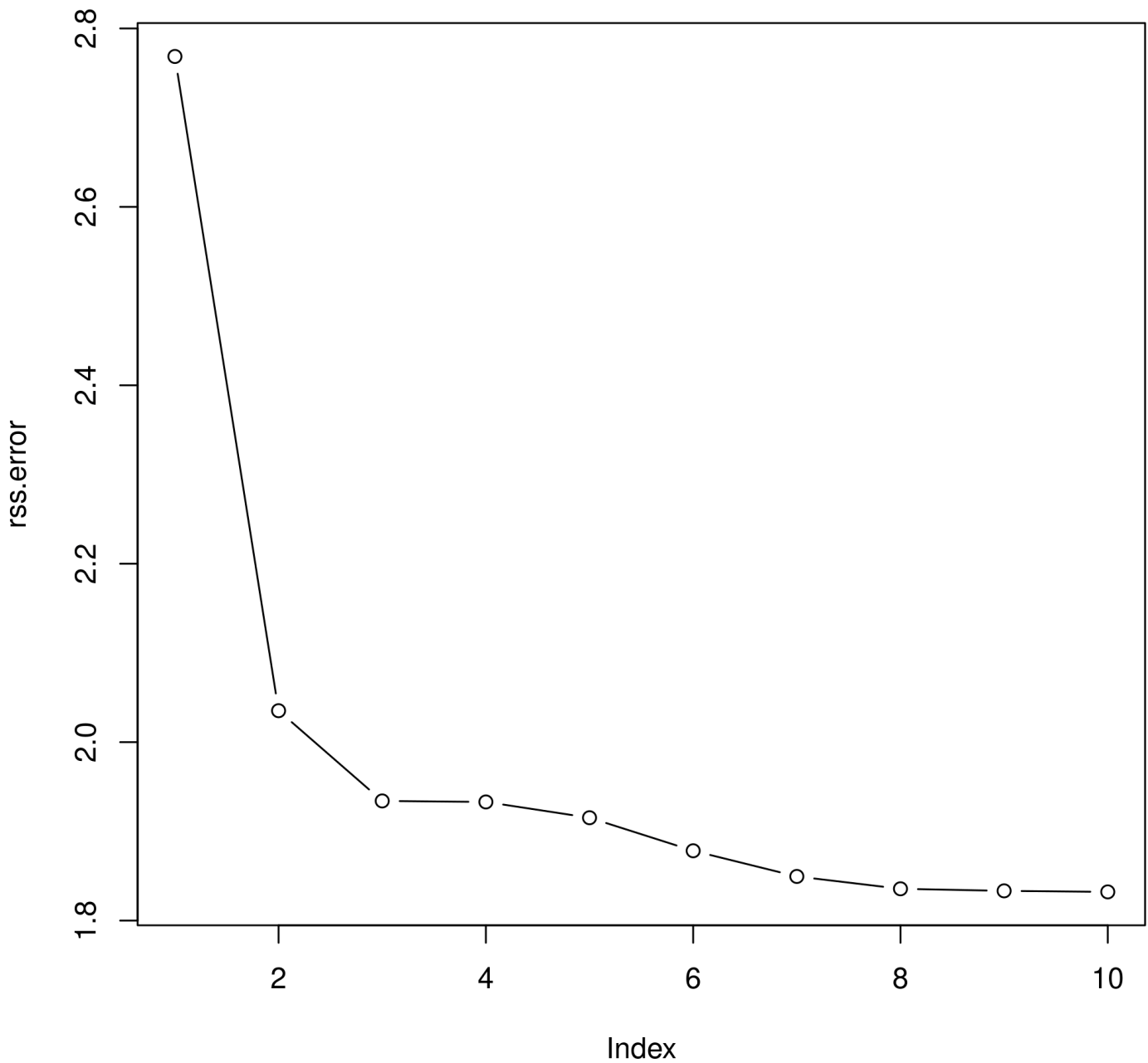
```

```

## [1] 2.768563 2.035262 1.934107 1.932981 1.915290 1.878257 1.849484 1.835630
## [9] 1.833331 1.832171

```

```
plot(rss.error, type="b")
```



c) Cross validation and polynomial selection

```

require(boot)
set.seed(1)
cv.error <- rep(0,10)
for (i in 1:10) {
  glm.fit <- glm(nox~poly(dis,i), data=boston)
  cv.error[i] <- cv.glm(boston, glm.fit, K=10)$delta[1] # [1]:std, [2]:bias-corrected
}
cv.error

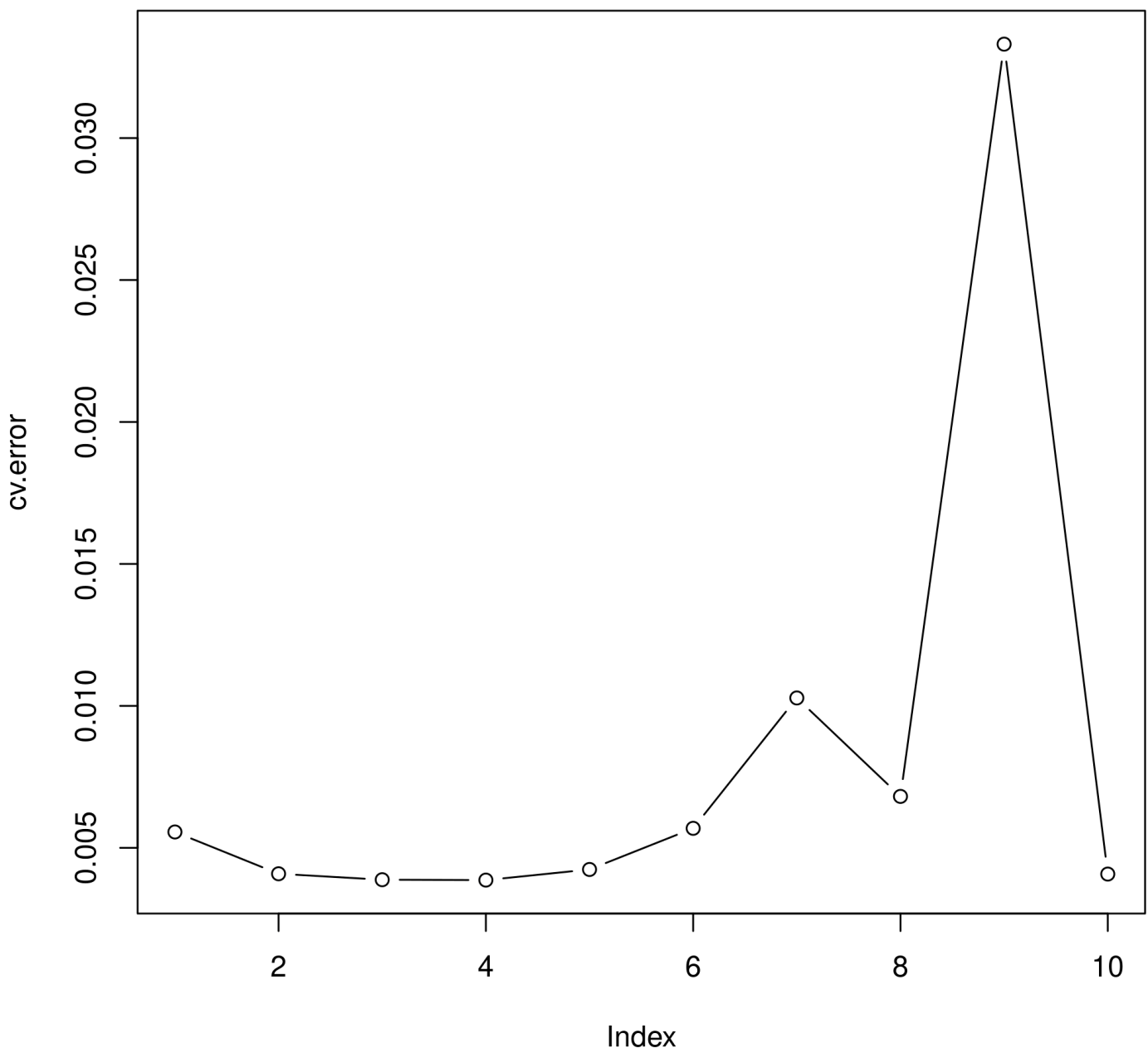
```

```

## [1] 0.005558263 0.004085706 0.003876521 0.003863342 0.004237452 0.005686862
## [7] 0.010278897 0.006810868 0.033308607 0.004075599

```

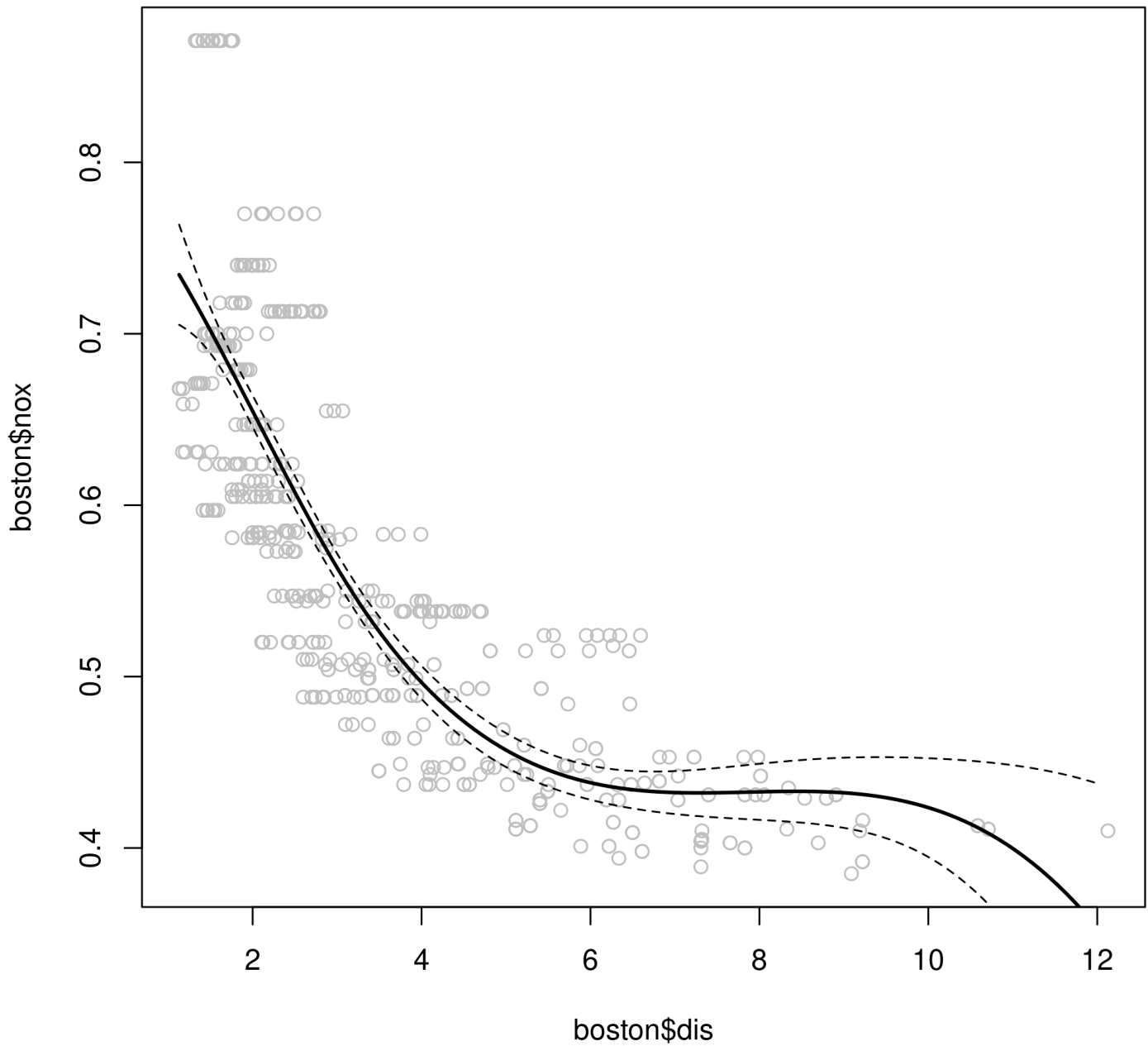
```
plot(cv.error, type="b")
```



- I feel like the second degree fit would be the most reasonable, though the fourth degree seems to be doing well.

d) Regression spline

```
fit.sp <- lm(nox~bs(dis, df=4), data=boston)
pred <- predict(fit.sp, newdata=list(dis=dis.grid), se=T)
plot(boston$dis, boston$nox, col="gray")
lines(dis.grid, pred$fit, lwd=2)
lines(dis.grid, pred$fit+2*pred$se, lty="dashed")
lines(dis.grid, pred$fit-2*pred$se, lty="dashed")
```



```
# set df to select knots at uniform quantiles of `dis`
attr(bs(boston$dis,df=4),"knots") # only 1 knot at 50th percentile
```

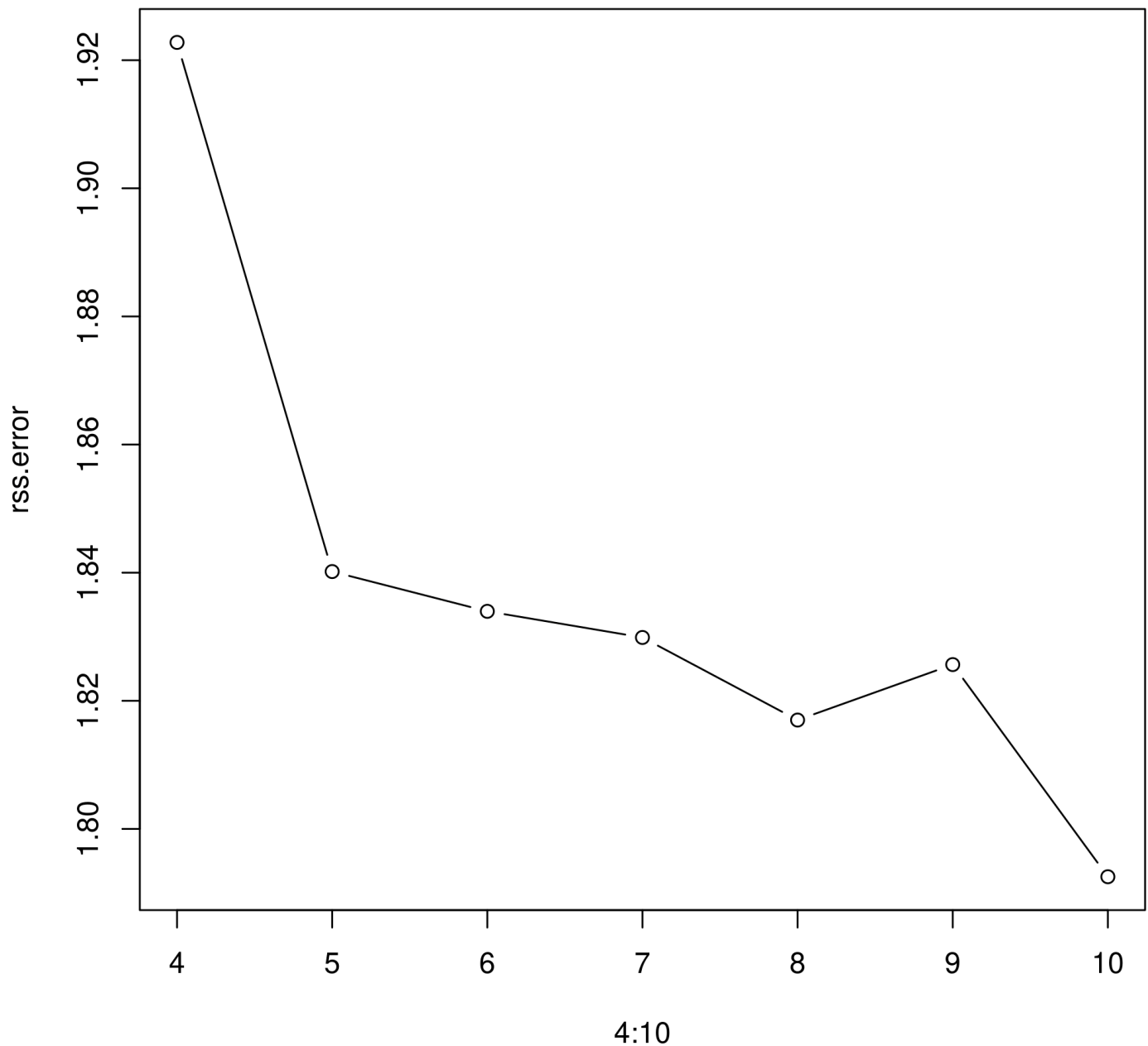
```
##      50%
## 3.20745
```

e) Range of regression splines

```
rss.error <- rep(0,7)
for (i in 4:10) {
  fit.sp <- lm(nox~bs(dis, df=i), data=boston)
  rss.error[i-3] <- sum(fit.sp$residuals^2)
}
rss.error
```

```
## [1] 1.922775 1.840173 1.833966 1.829884 1.816995 1.825653 1.792535
```

```
plot(4:10, rss.error, type="b")
```



- As the model gains more degrees of freedom, it tends to over fit to the training data better

f) Cross validation for best spline

```
cv.error <- rep(0,7)
for (i in 4:10) {
  glm.fit <- glm(nox~bs(dis, df=i), data=boston)
  cv.error[i-3] <- cv.glm(boston, glm.fit, K=10)$delta[1]
}
```

```
## Warning in bs(dis, degree = 3L, knots = c(`50%` = 3.1523), Boundary.knots =
## c(1.1296, : some 'x' values beyond boundary knots may cause ill-conditioned
## bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`50%` = 3.1523), Boundary.knots =
## c(1.1296, : some 'x' values beyond boundary knots may cause ill-conditioned
## bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`50%` = 3.2157), Boundary.knots =
## c(1.137, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`50%` = 3.2157), Boundary.knots =
## c(1.137, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```



```
## Warning in bs(dis, degree = 3L, knots = c(`33.33333` = 2.35953333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`33.33333` = 2.35953333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`33.33333` = 2.38403333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`33.33333` = 2.38403333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`25` = 2.07945, `50` = 3.1323, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`25` = 2.07945, `50` = 3.1323, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`25` = 2.1103, `50` = 3.2797, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`25` = 2.1103, `50` = 3.2797, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`20` = 1.9682, `40` = 2.7147, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`20` = 1.9682, `40` = 2.7147, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`20` = 1.95434, `40` = 2.59666, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`20` = 1.95434, `40` = 2.59666, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`16.66667` = 1.82203333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`16.66667` = 1.82203333333333, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`16.66667` = 1.8226, `33.33333` =
## 2.3817, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`16.66667` = 1.8226, `33.33333` =
## 2.3817, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`14.28571` = 1.7936, `28.57143`
## = 2.16972857142857, : some 'x' values beyond boundary knots may cause ill-
## conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`14.28571` = 1.7936, `28.57143`
## = 2.16972857142857, : some 'x' values beyond boundary knots may cause ill-
## conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`12.5` = 1.754625, `25` = 2.10215, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`12.5` = 1.754625, `25` = 2.10215, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

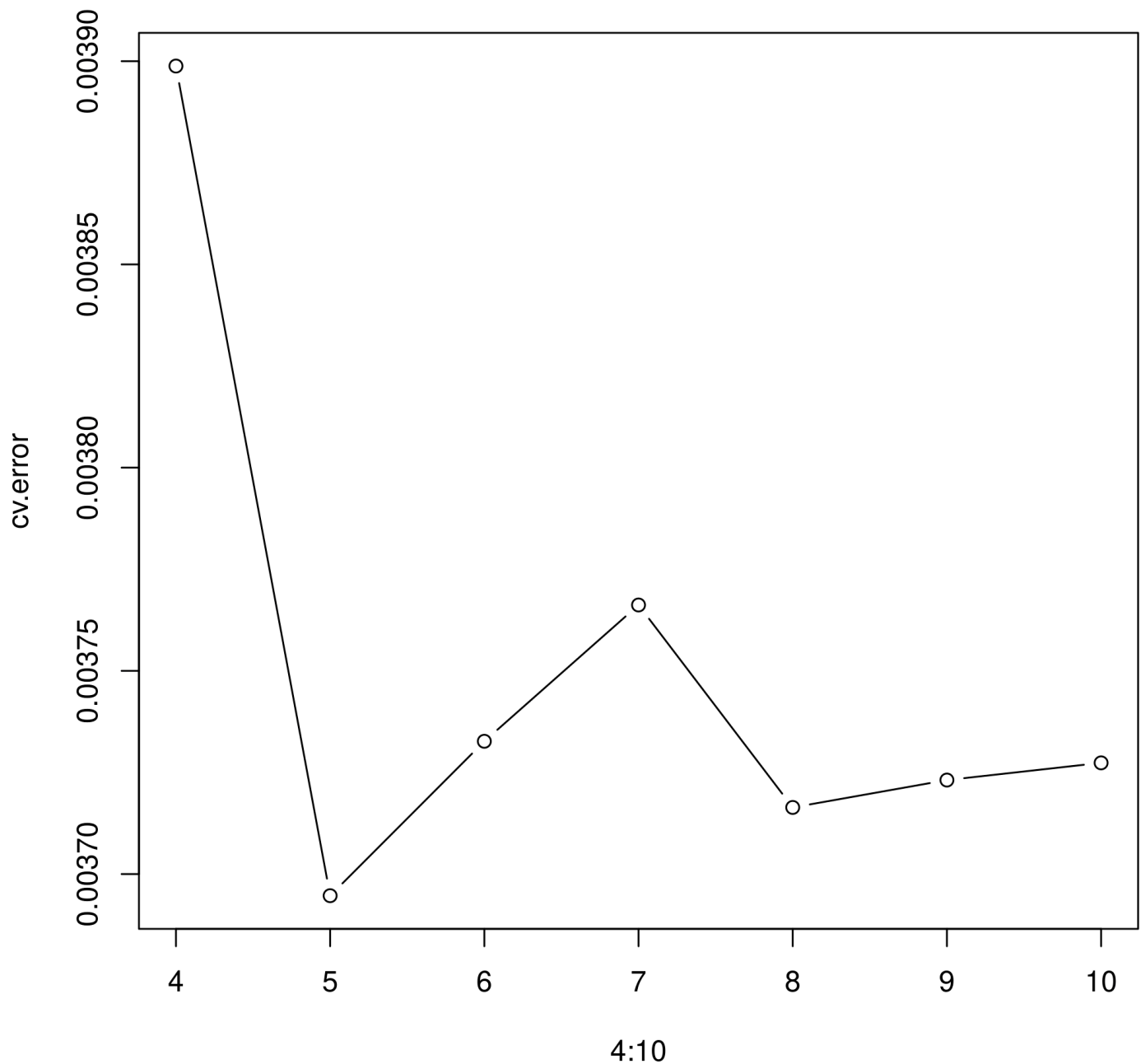
```
## Warning in bs(dis, degree = 3L, knots = c(`12.5` = 1.751575, `25` = 2.08755, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
## Warning in bs(dis, degree = 3L, knots = c(`12.5` = 1.751575, `25` = 2.08755, :
## some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
cv.error
```

```
## [1] 0.003898810 0.003694675 0.003732665 0.003766202 0.003716389 0.003723126
## [7] 0.003727358
```

```
plot(4:10, cv.error, type="b")
```



- A fifth degree polynomial is clearly indicated

Question 10 - Page 300

This question relates to the `College` data set.

- Split the data into a training set and a test set. Using out-of-state tuition as the response and the other variables as the predictors, perform forward stepwise selection on the training set in order to identify a satisfactory model that uses just a subset of the predictors.
- Fit a GAM on the training data, using out-of-state tuition as the response and the features selected in the previous step as the predictors. Plot the results, and explain your findings.
- Evaluate the model obtained on the test set, and explain the results obtained.
- For which variables, if any, is there evidence of a non-linear relationship with the response?

Answer

```
colDat<-ISLR::College  
colDat %>% str %>% print
```

```
## 'data.frame':   777 obs. of  18 variables:
## $ Private      : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 2 2 2 2 ...
## $ Apps         : num  1660 2186 1428 417 193 ...
## $ Accept       : num  1232 1924 1097 349 146 ...
## $ Enroll       : num   721 512 336 137 55 158 103 489 227 172 ...
## $ Top10perc    : num    23 16 22 60 16 38 17 37 30 21 ...
## $ Top25perc    : num    52 29 50 89 44 62 45 68 63 44 ...
## $ F.Undergrad  : num  2885 2683 1036 510 249 ...
## $ P.Undergrad  : num   537 1227 99 63 869 ...
## $ Outstate     : num  7440 12280 11250 12960 7560 ...
## $ Room.Board   : num  3300 6450 3750 5450 4120 ...
## $ Books        : num   450 750 400 450 800 500 500 450 300 660 ...
## $ Personal     : num  2200 1500 1165 875 1500 ...
## $ PhD          : num    70 29 53 92 76 67 90 89 79 40 ...
## $ Terminal     : num    78 30 66 97 72 73 93 100 84 41 ...
## $ S.F.Ratio    : num   18.1 12.2 12.9 7.7 11.9 9.4 11.5 13.7 11.3 11.5 ...
## $ perc.alumni  : num    12 16 30 37 2 11 26 37 23 15 ...
## $ Expend       : num  7041 10527 8735 19016 10922 ...
## $ Grad.Rate    : num    60 56 54 59 15 55 63 73 80 52 ...
## NULL
```

```
colDat %>% summary %>% print
```

```
## Private      Apps      Accept      Enroll      Top10perc
## No :212      Min.   : 81      Min.   : 72      Min.   : 35      Min.   : 1.00
## Yes:565      1st Qu.: 776      1st Qu.: 604      1st Qu.: 242      1st Qu.:15.00
##           Median : 1558      Median : 1110      Median : 434      Median :23.00
##           Mean   : 3002      Mean   : 2019      Mean   : 780      Mean   :27.56
##           3rd Qu.: 3624      3rd Qu.: 2424      3rd Qu.: 902      3rd Qu.:35.00
##           Max.   :48094      Max.   :26330      Max.   :6392      Max.   :96.00
## Top25perc    F.Undergrad  P.Undergrad      Outstate
## Min.   : 9.0      Min.   : 139      Min.   : 1.0      Min.   : 2340
## 1st Qu.: 41.0      1st Qu.: 992      1st Qu.: 95.0      1st Qu.: 7320
## Median : 54.0      Median : 1707      Median : 353.0      Median : 9990
## Mean   : 55.8      Mean   : 3700      Mean   : 855.3      Mean   :10441
## 3rd Qu.: 69.0      3rd Qu.: 4005      3rd Qu.: 967.0      3rd Qu.:12925
## Max.   :100.0      Max.   :31643      Max.   :21836.0      Max.   :21700
## Room.Board   Books      Personal      PhD
## Min.   :1780      Min.   : 96.0      Min.   : 250      Min.   : 8.00
## 1st Qu.:3597      1st Qu.: 470.0      1st Qu.: 850      1st Qu.: 62.00
## Median :4200      Median : 500.0      Median :1200      Median : 75.00
## Mean   :4358      Mean   : 549.4      Mean   :1341      Mean   : 72.66
## 3rd Qu.:5050      3rd Qu.: 600.0      3rd Qu.:1700      3rd Qu.: 85.00
## Max.   :8124      Max.   :2340.0      Max.   :6800      Max.   :103.00
## Terminal     S.F.Ratio    perc.alumni      Expend
## Min.   : 24.0      Min.   : 2.50      Min.   : 0.00      Min.   : 3186
## 1st Qu.: 71.0      1st Qu.:11.50      1st Qu.:13.00      1st Qu.: 6751
## Median : 82.0      Median :13.60      Median :21.00      Median : 8377
## Mean   : 79.7      Mean   :14.09      Mean   :22.74      Mean   : 9660
## 3rd Qu.: 92.0      3rd Qu.:16.50      3rd Qu.:31.00      3rd Qu.:10830
## Max.   :100.0      Max.   :39.80      Max.   :64.00      Max.   :56233
## Grad.Rate
## Min.   : 10.00
## 1st Qu.: 53.00
## Median : 65.00
## Mean   : 65.46
## 3rd Qu.: 78.00
## Max.   :118.00
```

```
colDat %>% sapply(unique) %>% sapply(length) %>% print
```

```
## Private      Apps      Accept      Enroll      Top10perc      Top25perc
## 2           711           693           581           82           89
## F.Undergrad P.Undergrad      Outstate      Room.Board      Books      Personal
## 714           566           640           553           122           294
## PhD      Terminal      S.F.Ratio    perc.alumni      Expend      Grad.Rate
## 78           65           173           61           744           81
```

```
plotLEAP=function(leapObj){
  par(mfrow = c(2,2))
  bar2=which.max(leapObj$adjr2)
  bbic=which.min(leapObj$bic)
  bcp=which.min(leapObj$cp)
  plot(leapObj$rss,xlab="Number of variables",ylab="RSS",type="b")
  plot(leapObj$adjr2,xlab="Number of variables",ylab=TeX("Adjusted R^2"),type="b")
  points(bar2, leapObj$adjr2[bar2],col="green",cex=2,pch=20)
  plot(leapObj$bic,xlab="Number of variables",ylab=TeX("BIC"),type="b")
  points(bbic, leapObj$bic[bbic],col="blue",cex=2,pch=20)
  plot(leapObj$cp,xlab="Number of variables",ylab=TeX("C_p"),type="b")
  points(bcp, leapObj$cp[bcp],col="red",cex=2,pch=20)
}
```

a) Train test

```
train_ind = sample(colDat %>% nrow,100)
test_ind = setdiff(seq_len(colDat %>% nrow), train_ind)
```

Best subset selection

```
train_set<-colDat[train_ind,]  
test_set<-colDat[-train_ind,]
```

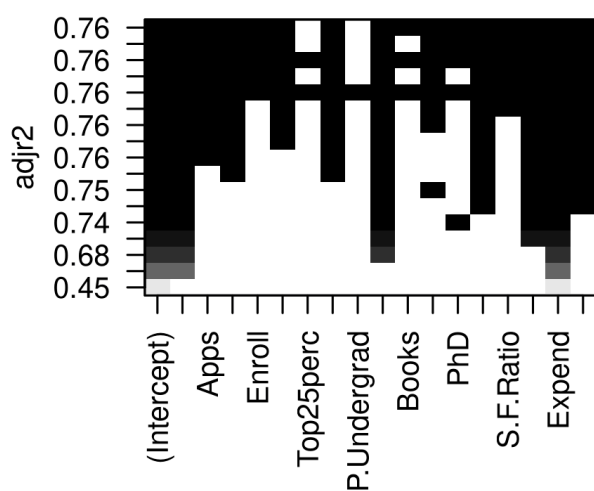
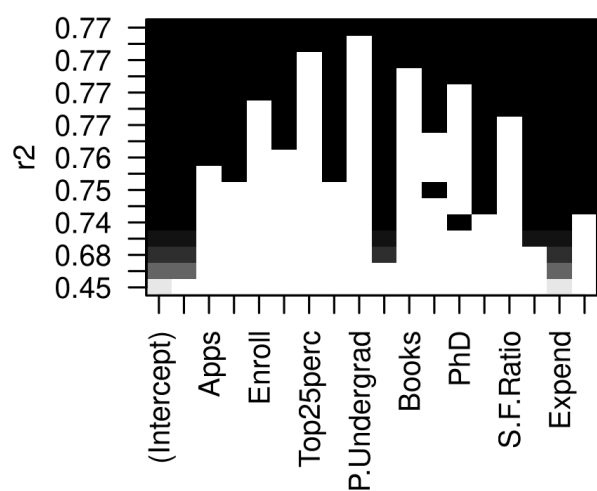
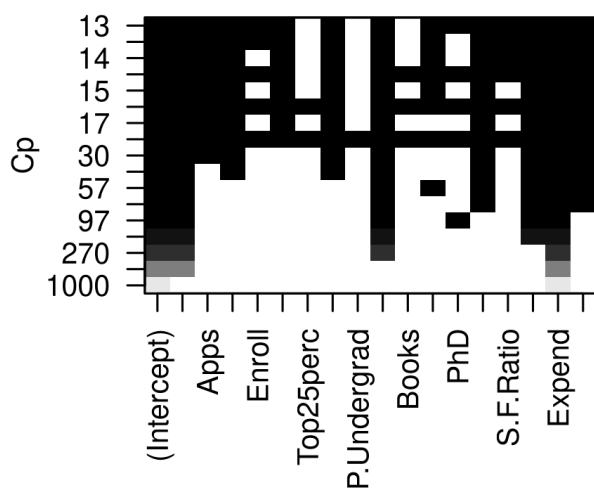
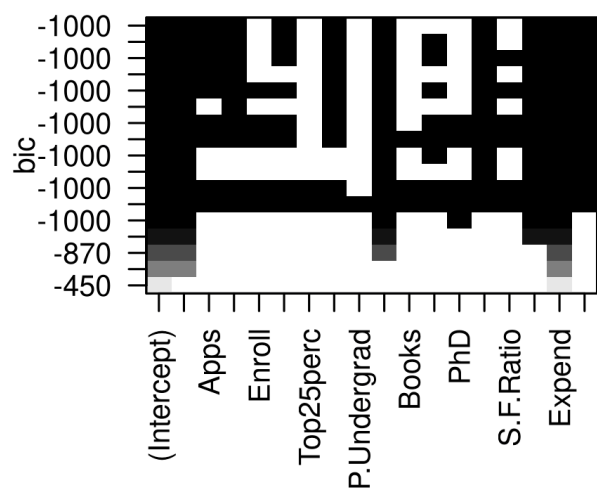
```
library(leaps)
```

```
modelFit<-regsubsets(Outstate~.,data=colDat,nvmax=20)  
modelFit %>% summary %>% print
```

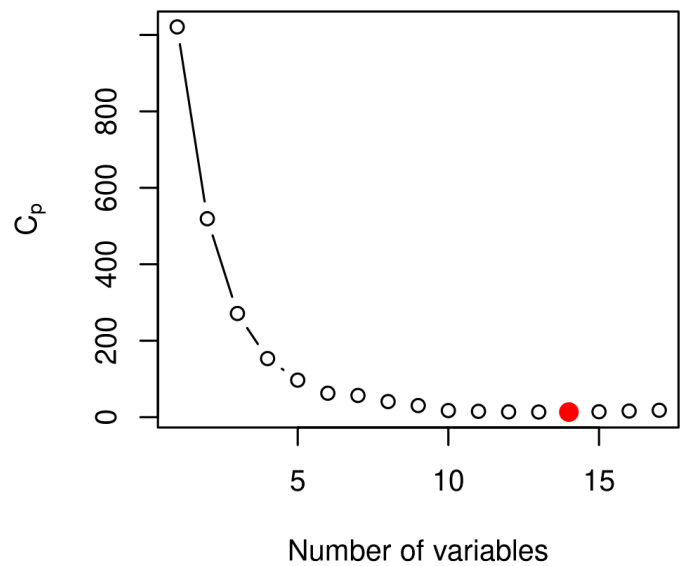
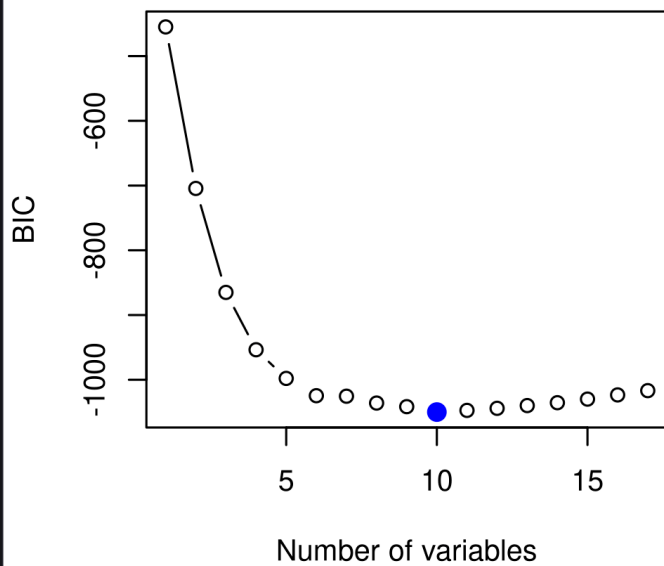
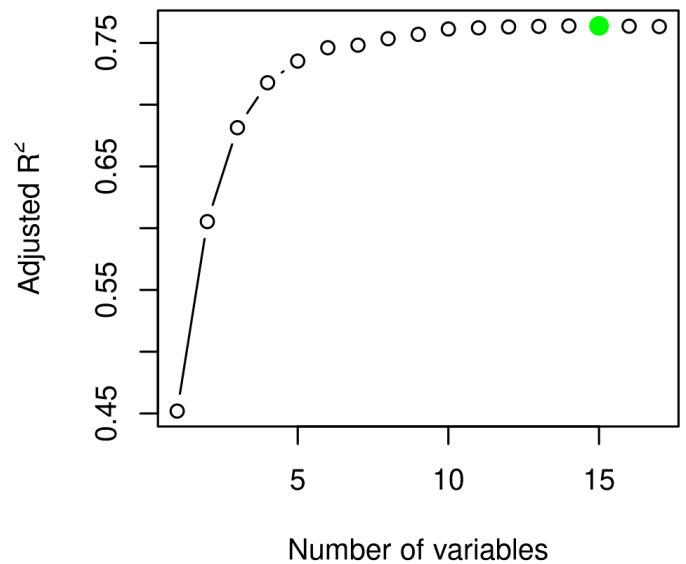
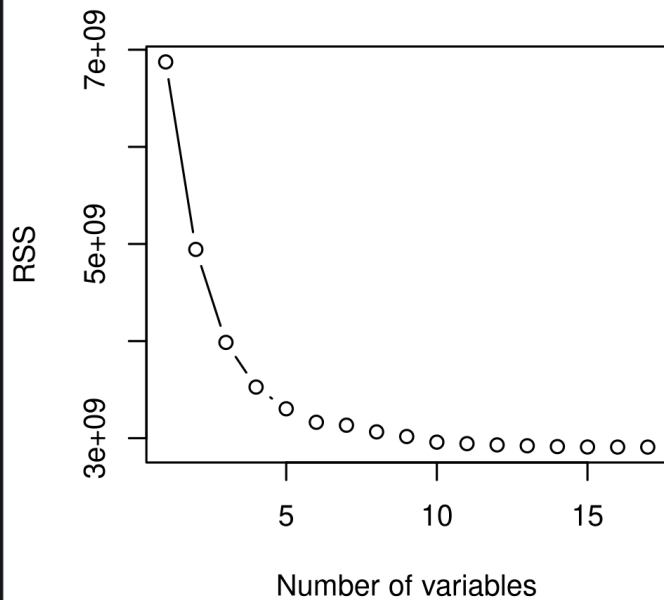
```
## Subset selection object  
## Call: regsubsets.formula(Outstate ~ ., data = colDat, nvmax = 20)  
## 17 Variables (and intercept)  
##           Forced in Forced out  
## PrivateYes      FALSE      FALSE  
## Apps            FALSE      FALSE  
## Accept          FALSE      FALSE  
## Enroll          FALSE      FALSE  
## Top10perc       FALSE      FALSE  
## Top25perc       FALSE      FALSE  
## F.Undergrad     FALSE      FALSE  
## P.Undergrad     FALSE      FALSE  
## Room.Board      FALSE      FALSE  
## Books           FALSE      FALSE  
## Personal        FALSE      FALSE  
## PhD             FALSE      FALSE  
## Terminal        FALSE      FALSE  
## S.F.Ratio       FALSE      FALSE  
## perc.alumni     FALSE      FALSE  
## Expend          FALSE      FALSE  
## Grad.Rate       FALSE      FALSE  
## 1 subsets of each size up to 17  
## Selection Algorithm: exhaustive  
##           PrivateYes Apps Accept Enroll Top10perc Top25perc F.Undergrad  
## 1 ( 1 ) " " " " " " " " " "  
## 2 ( 1 ) " " " " " " " " " "  
## 3 ( 1 ) " " " " " " " " " "  
## 4 ( 1 ) " " " " " " " " " "  
## 5 ( 1 ) " " " " " " " " " "  
## 6 ( 1 ) " " " " " " " " " "  
## 7 ( 1 ) " " " " " " " " " "  
## 8 ( 1 ) " " " * " " " " " "  
## 9 ( 1 ) " " " " * * " " " " "  
## 10 ( 1 ) " " " " * * " " " " "  
## 11 ( 1 ) " " " " * * " " " " "  
## 12 ( 1 ) " " " " * * " " " " "  
## 13 ( 1 ) " " " " * * " " " " "  
## 14 ( 1 ) " " " " * * " " " " "  
## 15 ( 1 ) " " " " * * " " " " "  
## 16 ( 1 ) " " " " * * " " " " "  
## 17 ( 1 ) " " " " * * " " " " "  
##           P.Undergrad Room.Board Books Personal PhD Terminal S.F.Ratio  
## 1 ( 1 ) " " " " " " " " " "  
## 2 ( 1 ) " " " " " " " " " "  
## 3 ( 1 ) " " " * " " " " " "  
## 4 ( 1 ) " " " * " " " " " "  
## 5 ( 1 ) " " " * " " " " " "  
## 6 ( 1 ) " " " * " " " " " "  
## 7 ( 1 ) " " " * " " " " " "  
## 8 ( 1 ) " " " * " " " " " "  
## 9 ( 1 ) " " " * " " " " " "  
## 10 ( 1 ) " " " * " " " " " "  
## 11 ( 1 ) " " " * " " " " " "  
## 12 ( 1 ) " " " * " " " " " "  
## 13 ( 1 ) " " " * " " " " " "  
## 14 ( 1 ) " " " * " " " " " "  
## 15 ( 1 ) " " " * " " " " " "  
## 16 ( 1 ) " " " * " " " " " "  
## 17 ( 1 ) " " " * " " " " " "  
##           perc.alumni Expend Grad.Rate  
## 1 ( 1 ) " " " * " " "  
## 2 ( 1 ) " " " * " " "  
## 3 ( 1 ) " " " * " " "  
## 4 ( 1 ) " " " * " " "  
## 5 ( 1 ) " " " * " " "  
## 6 ( 1 ) " " " * " " "  
## 7 ( 1 ) " " " * " " "  
## 8 ( 1 ) " " " * " " "  
## 9 ( 1 ) " " " * " " "  
## 10 ( 1 ) " " " * " " "  
## 11 ( 1 ) " " " * " " "  
## 12 ( 1 ) " " " * " " "  
## 13 ( 1 ) " " " * " " "  
## 14 ( 1 ) " " " * " " "  
## 15 ( 1 ) " " " * " " "  
## 16 ( 1 ) " " " * " " "  
## 17 ( 1 ) " " " * " " "
```

We might want to take a look at these.

```
par(mfrow=c(2,2))  
plot(modelFit)  
plot(modelFit,scale='Cp')  
plot(modelFit,scale='r2')  
plot(modelFit,scale='adjr2')
```



```
plotLEAP(modelFit %>% summary)
```



- So we like 14 variables, namely

```
coefficients(modelFit,id=14) %>% print
```

```
## (Intercept) PrivateYes Apps Accept Enroll
## -1.817040e+03 2.256946e+03 -2.999022e-01 8.023519e-01 -5.372545e-01
## Top10perc F.Undergrad Room.Board Personal PhD
## 2.365529e+01 -9.569936e-02 8.741819e-01 -2.478418e-01 1.269506e+01
## Terminal S.F.Ratio perc.alumni Expend Grad.Rate
## 2.297296e+01 -4.700560e+01 4.195006e+01 2.003912e-01 2.383197e+01
```

- But five seems like a better bet.

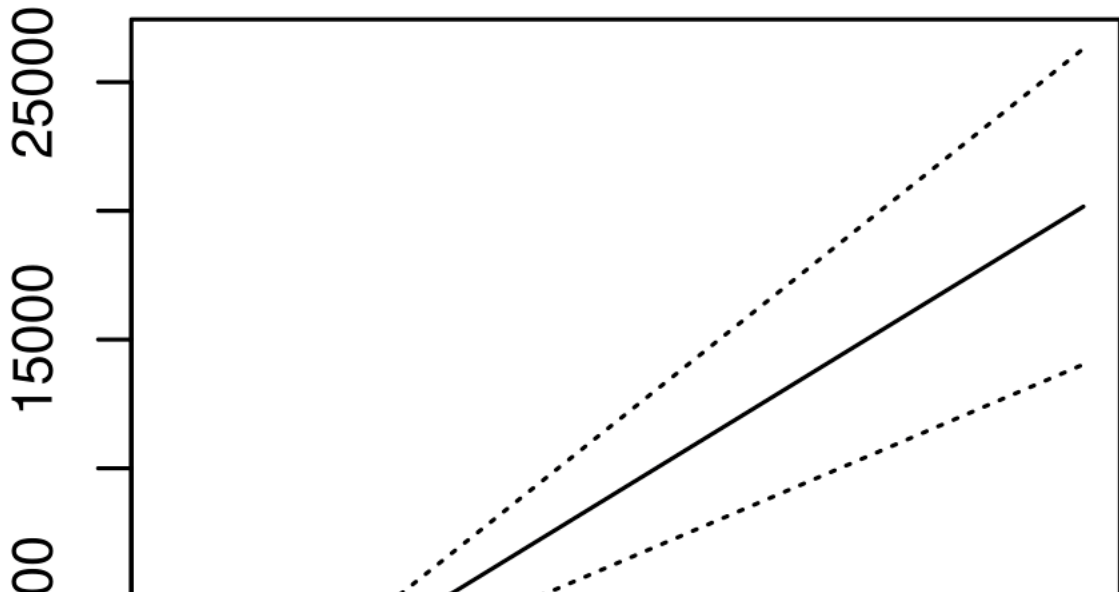
```
coefficients(modelFit,id=5)
```

```
## (Intercept) PrivateYes Room.Board PhD perc.alumni
## -2864.6325619 2936.7416766 1.0677573 40.5334088 61.3147684
## Expend
## 0.2253945
```

b) GAM

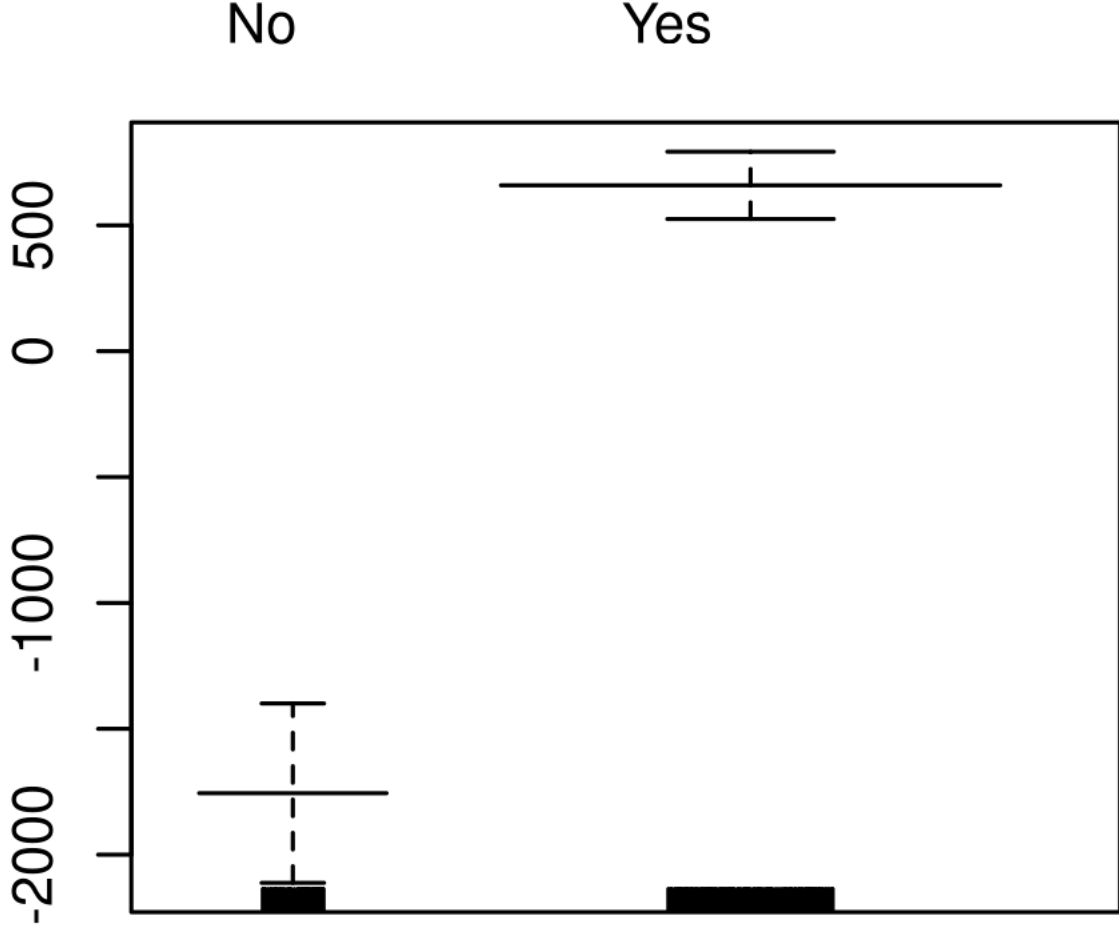
```
library(gam)
```


partial for Accept

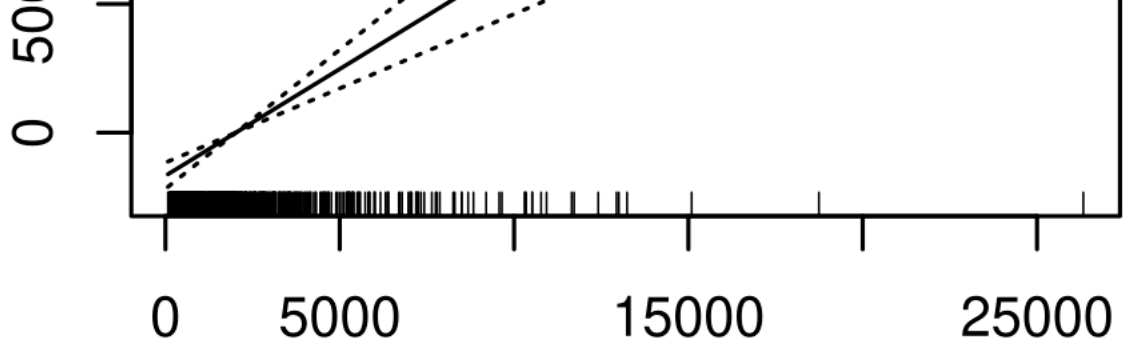


partial for Enroll

partial for Private

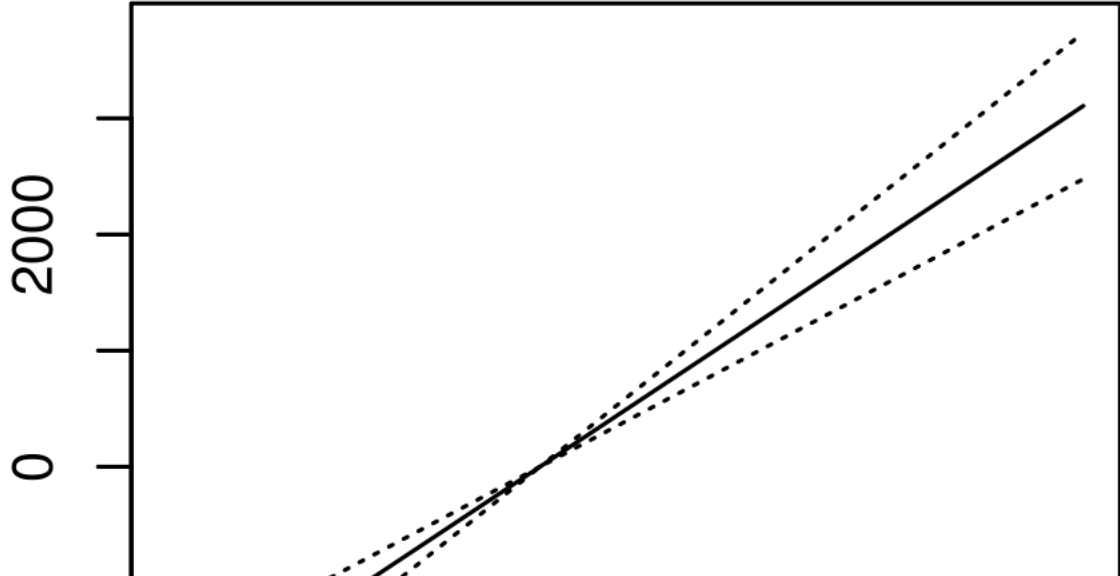


s(Apps, 3)



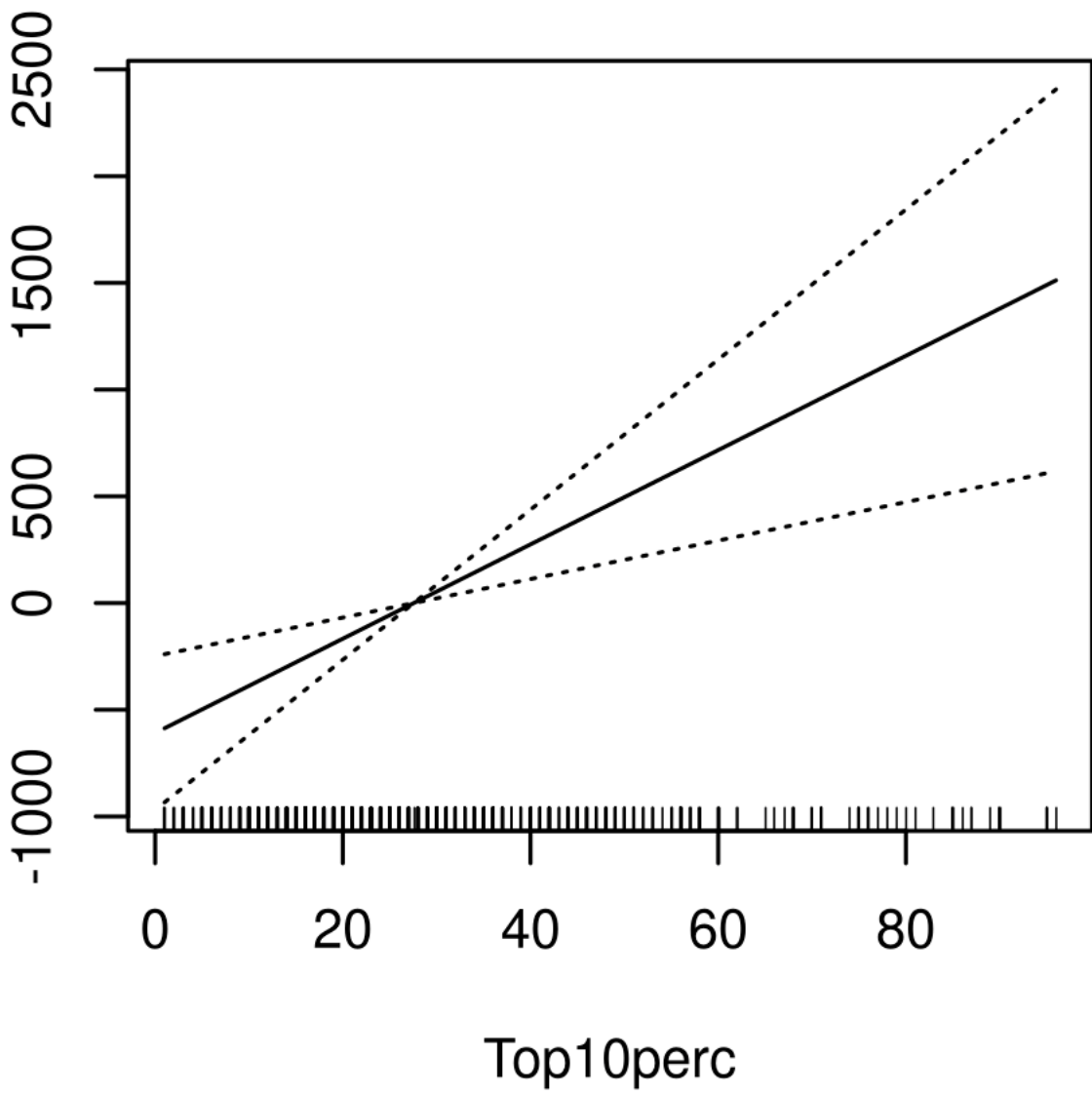
Accept

partial for Room.Board



partial for Personal

partial for Top10perc



partial for F.Undergrad

part

-2000

2000

4000

6000

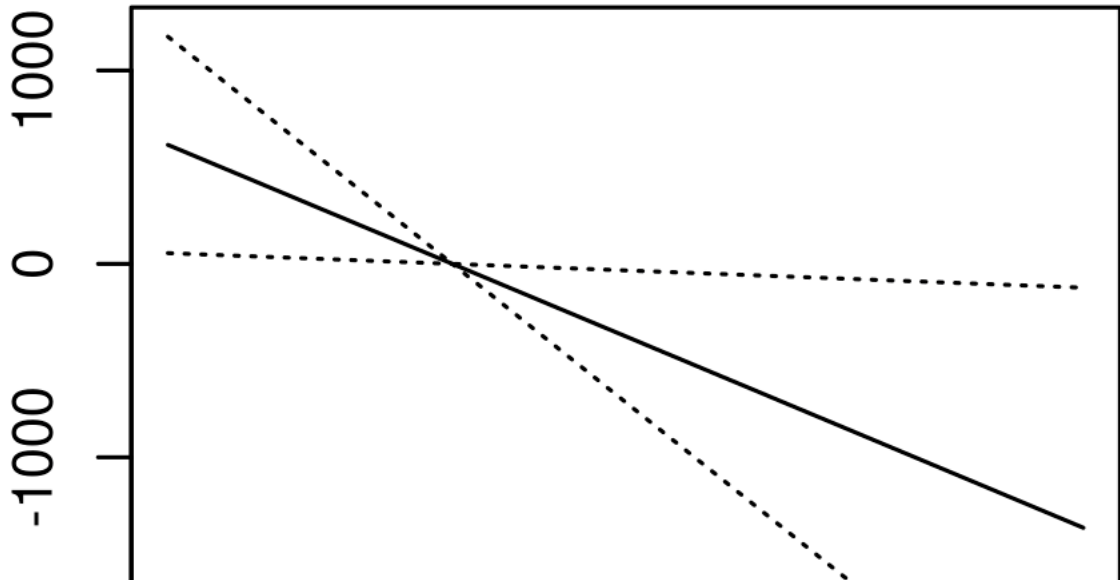
8000

Room.Board

pa

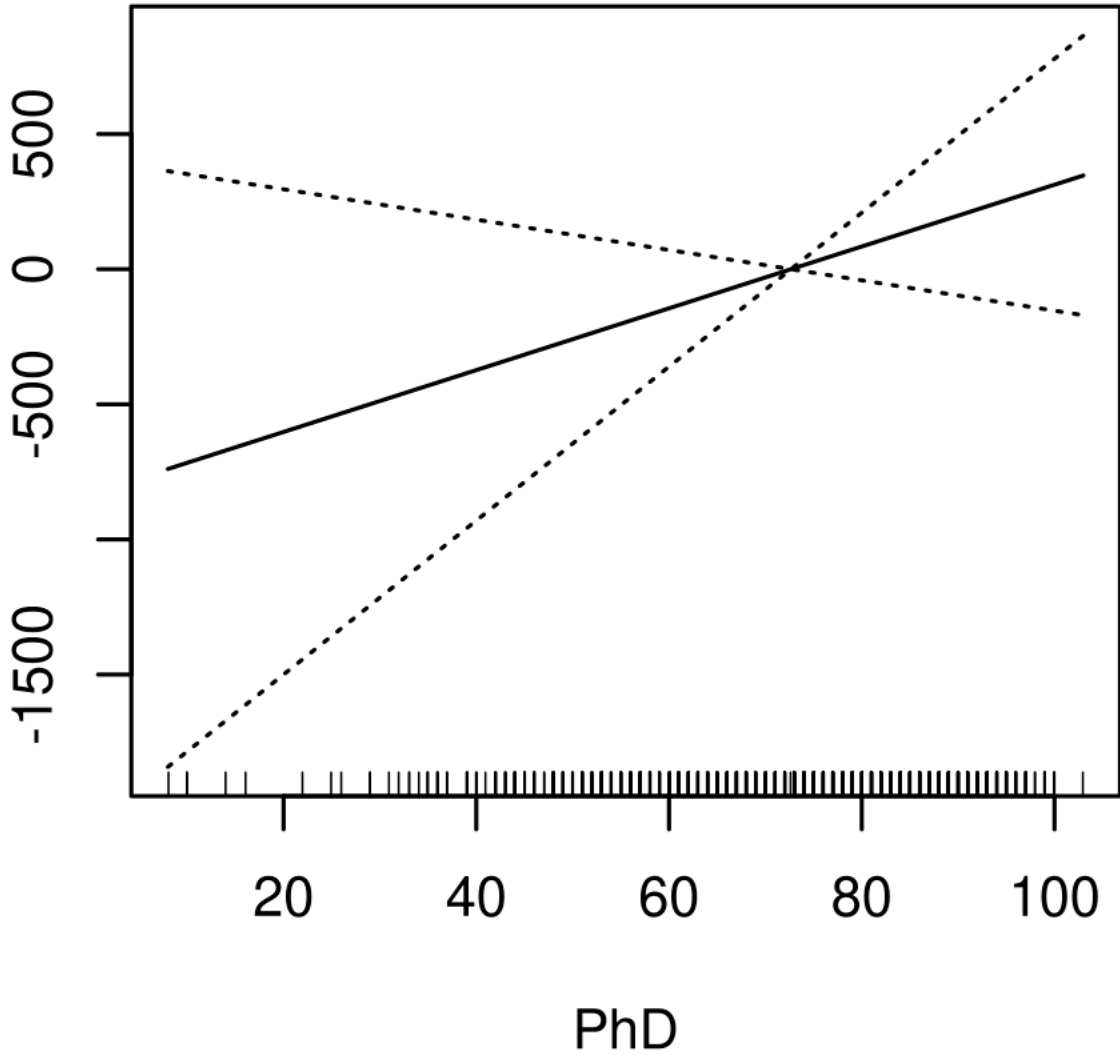


partial for S.F.Ratio

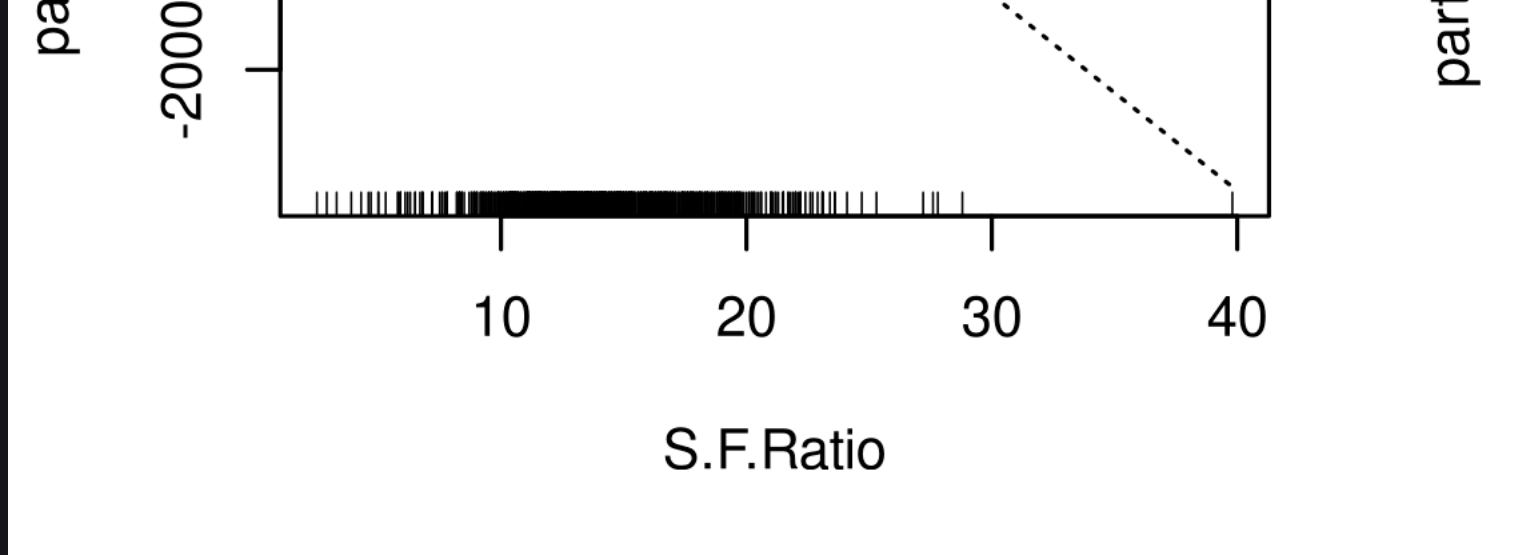


partial for perc.alumni

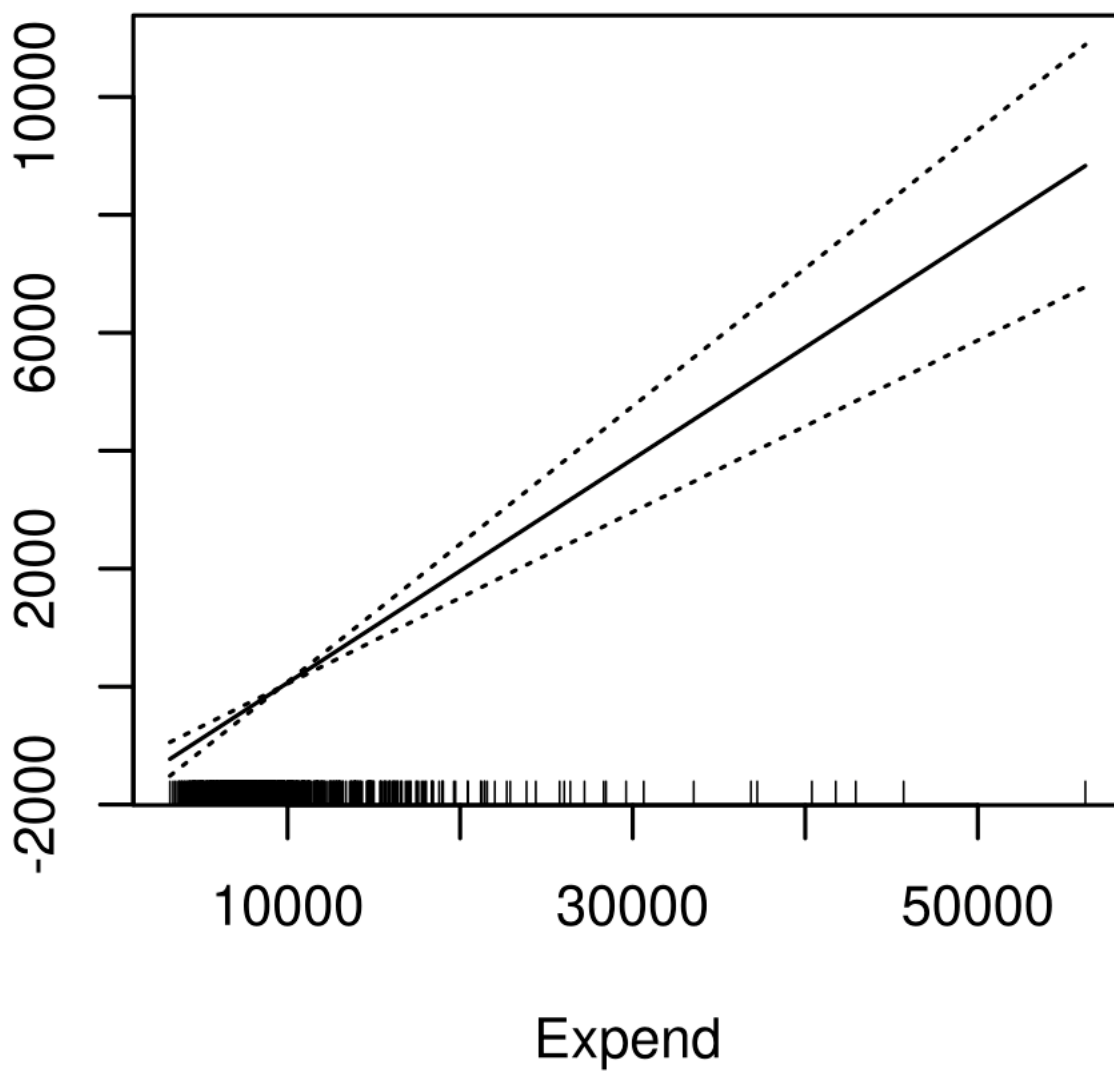
partial for PhD



partial for Terminal

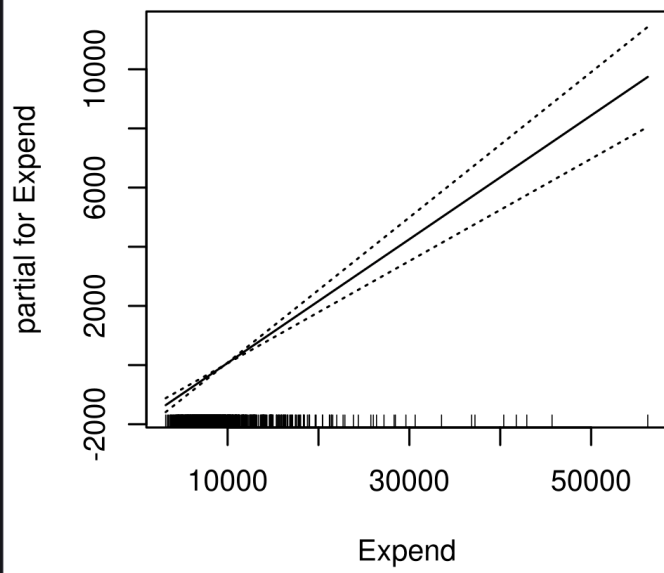


partial for Expend



partial for Grad.Rate

```
par(mfrow=c(2,2))  
plot(fit2,se=TRUE)
```



```
pred <- predict(fit, test_set)
mse.error <- mean((test_set$Outstate - pred)^2)
mse.error %>% print
```

```
## [1] 3691891
```

```
gam.tss = mean((test_set$Outstate - mean(test_set$Outstate))^2)
test.rss = 1 - mse.error/gam.tss
test.rss %>% print
```

```
## [1] 0.7731239
```

```
pred2 <- predict(fit2, test_set)
mse.error2 <- mean((test_set$Outstate - pred2)^2)
mse.error2 %>% print
```

```
## [1] 4121902
```

```
gam.tss2 = mean((test_set$Outstate - mean(test_set$Outstate))^2)
test.rss2 = 1 - mse.error2/gam.tss2
test.rss2 %>% print
```

```
## [1] 0.7466987
```

This is pretty good model, all told.

d) Summary

```
summary(fit) %>% print
```

```
##
## Call: gam(formula = Outstate ~ Private + s(Apps, 3) + Accept + Enroll +
##       Top10perc + F.Undergrad + Room.Board + Personal + PhD + Terminal +
##       S.F.Ratio + perc.alumni + Expend + Grad.Rate, data = colDat)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -6641.083 -1262.806   -5.698  1270.911  9965.901
##
## (Dispersion Parameter for gaussian family taken to be 3749048)
##
##      Null Deviance: 12559297426 on 776 degrees of freedom
## Residual Deviance: 2849276343 on 760 degrees of freedom
## AIC: 13985.3
##
## Number of Local Scoring Iterations: 2
##
## Anova for Parametric Effects
##      Df      Sum Sq    Mean Sq  F value    Pr(>F)
## Private      1  4034912907  4034912907  1076.250 < 2.2e-16 ***
## s(Apps, 3)    1  1344548030  1344548030   358.637 < 2.2e-16 ***
## Accept       1   90544274   90544274    24.151 1.091e-06 ***
## Enroll       1  144471570  144471570    38.535 8.838e-10 ***
## Top10perc    1  1802244831  1802244831   480.721 < 2.2e-16 ***
## F.Undergrad  1   45230645   45230645    12.065 0.0005430 ***
## Room.Board   1  1110285773  1110285773   296.151 < 2.2e-16 ***
## Personal     1   47886988   47886988    12.773 0.0003738 ***
## PhD          1  220249039  220249039    58.748 5.476e-14 ***
## Terminal     1   66366007   66366007    17.702 2.892e-05 ***
## S.F.Ratio    1  190811028  190811028    50.896 2.274e-12 ***
## perc.alumni  1  225293653  225293653    60.094 2.904e-14 ***
## Expend       1  258162295  258162295    68.861 4.805e-16 ***
## Grad.Rate    1   57947219   57947219    15.457 9.214e-05 ***
## Residuals    760 2849276343    3749048
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##      Npar Df Npar F      Pr(F)
## (Intercept)
## Private
## s(Apps, 3)      2  8.571 0.0002085 ***
## Accept
## Enroll
## Top10perc
## F.Undergrad
## Room.Board
## Personal
## PhD
## Terminal
## S.F.Ratio
## perc.alumni
## Expend
## Grad.Rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```



```
summary(fit2) %>% print
```

```
##
## Call: gam(formula = Outstate ~ Private + s(Room.Board, 2) + s(PhD,
##      3) + s(perc.alumni) + Expend, data = colDat)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -8676.030 -1345.678   -8.409  1265.524  9590.459
##
## (Dispersion Parameter for gaussian family taken to be 4175193)
##
##      Null Deviance: 12559297426 on 776 degrees of freedom
## Residual Deviance: 3194023899 on 765.0002 degrees of freedom
## AIC: 14064.05
##
## Number of Local Scoring Iterations: 2
##
## Anova for Parametric Effects
##      Df      Sum Sq   Mean Sq F value    Pr(>F)
## Private      1 3751107814 3751107814   898.43 < 2.2e-16 ***
## s(Room.Board, 2) 1 2913770756 2913770756   697.88 < 2.2e-16 ***
## s(PhD, 3)      1 1149711330 1149711330   275.37 < 2.2e-16 ***
## s(perc.alumni)  1  556759894  556759894   133.35 < 2.2e-16 ***
## Expend        1  554812125  554812125   132.88 < 2.2e-16 ***
## Residuals     765 3194023899    4175193
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##      Npar Df Npar F      Pr(F)
## (Intercept)
## Private
## s(Room.Board, 2)      1 4.9853 0.0258517 *
## s(PhD, 3)            2 9.1614 0.0001171 ***
## s(perc.alumni)       3 0.8726 0.4548496
## Expend
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Introduction to Statistical Learning: with Applications in R.
Berlin, Germany: Springer Science & Business Media.